

СОДЕРЖАНИЕ

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ	5
ВВЕДЕНИЕ	6
1 Исследовательская часть	8
1.1 Задача глобальной оптимизации и метаэвристики	8
1.2 Алгоритм биогеографической оптимизации	11
1.3 Модификации модели миграции	13
1.4 Модификации базового алгоритма	14
1.5 Метод роя частиц и сравнение с биогеографической оптими- зацией	16
1.6 Методы визуализации результатов оптимизации	17
1.7 Обзор программ-аналогов	19
2 Конструкторская часть	22
2.1 Функциональные требования к программному комплексу . . .	22
2.2 Архитектура программного комплекса	23
2.3 Проектирование протокола обмена данными	25
2.4 Модель хранения данных	27
2.5 Проектирование пользовательского интерфейса	29
3 Технологическая часть	31
3.1 Обоснование выбора технологий	31
3.2 Реализация вычислительного ядра сервера	33
3.3 Реализация очереди задач и обработчика	34
3.4 Реализация алгоритмов оптимизации	36
3.5 Реализация мобильного клиента	40
3.5.1 Клиент протокола	40
3.5.2 Модели данных и конверт сообщения	41
3.5.3 Сборка конфигурации задачи	42
3.5.4 Управление состоянием	43
3.5.5 Локальное хранение истории	44

3.6	Пользовательский интерфейс приложения	44
3.6.1	Подключение к серверу	44
3.6.2	Настройка эксперимента	45
3.6.3	Выполнение и наблюдение за прогрессом	46
3.6.4	Просмотр результата	47
3.6.5	Трёхмерная визуализация	49
3.6.6	История и сравнение запусков	50
3.7	Реализация визуализации результатов	51
3.7.1	График сходимости	51
3.7.2	Контурная карта	52
3.7.3	Трёхмерная поверхность	53
3.7.4	Анимация процесса оптимизации	53
4	Аналитическая часть	55
4.1	Тестирование программного обеспечения	55
4.1.1	Автоматическое тестирование серверной части	55
4.1.2	Функциональное тестирование клиента	56
4.2	Вычислительные эксперименты и анализ результатов	57
4.2.1	Методика экспериментов	58
4.2.2	Сравнение методов на функции с множеством минимумов	58
4.2.3	Сравнение методов на разных функциях	59
4.2.4	Выводы по результатам	60
	ЗАКЛЮЧЕНИЕ	62
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	64
	ПРИЛОЖЕНИЯ	67

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

В настоящей ВКР применяют следующие сокращения и обозначения.

BBO	— алгоритм биогеографической оптимизации (Biogeography-Based Optimization)
GIF	— формат обмена графическими данными (Graphics Interchange Format)
HSI	— индекс пригодности среды обитания (Habitat Suitability Index)
HTTP	— протокол передачи гипертекста (Hypertext Transfer Protocol)
JSON	— текстовый формат обмена данными (JavaScript Object Notation)
PSO	— метод роя частиц (Particle Swarm Optimization)
TCP	— протокол управления передачей (Transmission Control Protocol)
WebSocket	— протокол полнодуплексной связи поверх TCP

ВВЕДЕНИЕ

Многие практические задачи сводятся к поиску глобального минимума целевой функции. Часто у такой функции, помимо глобального минимума, есть множество локальных. Классические градиентные методы здесь малоэффективны: они сходятся к ближайшему локальному минимуму и не умеют из него выбираться. Поэтому для функций с множеством локальных минимумов применяют метаэвристические алгоритмы, которые работают сразу с целой популяцией решений [1]. Один из них — алгоритм биогеографической оптимизации (Biogeography-Based Optimization, далее — BBO) [2, 3].

Поведение метаэвристик трудно предсказать аналитически, поэтому об их работе чаще судят по результатам запусков. Здесь многое даёт визуализация: по ней видно, как убывает лучшее найденное значение, как популяция распределяется в пространстве решений и не наступает ли преждевременная сходимость. Удобно, когда такой инструмент доступен прямо на мобильном устройстве и не требует настройки вычислительного окружения. Этим и обусловлена актуальность работы.

Объект исследования — модификации алгоритма биогеографической оптимизации, а также метод роя частиц, который привлекается для сравнения. Программа рассчитана прежде всего на студентов, изучающих методы оптимизации, и на исследователей, которым нужно быстро сравнить поведение разных метаэвристик на тестовых задачах.

Цель работы — разработка программно-аппаратного комплекса поддержки мобильного приложения визуализации результатов работы модификаций эволюционного алгоритма оптимизации. Для её достижения решается несколько связанных между собой задач.

Вычисления выполняет серверное ядро. Оно должно поддерживать целое семейство методов: классический BBO, его варианты с синусоидальной и смешанной моделями миграции, их модификации с адаптивной мутацией, перезапуском при стагнации и локальным поиском, а также метод роя частиц. Запускать алгоритмы нужно как на стандартных тестовых функциях, так и на функции, которую пользователь задаёт собственным выражением.

Чтобы мобильное устройство могло ставить задачи серверу и получать от него данные, нужен протокол обмена. Он должен поддерживать постановку

задачи, передачу прогресса по итерациям в реальном времени, выдачу итогового результата, отмену задачи и работу очереди, когда клиентов несколько.

С пользователем взаимодействует мобильное приложение. Через него настраивают эксперимент, отправляют задачу на сервер и следят за ходом вычислений и итогом.

Главное в приложении — наглядно показать, как работает алгоритм. Для этого строится график сходимости в реальном времени, пространство поиска отображается контурной картой и трёхмерной поверхностью, а динамика популяции — анимацией.

Смысл комплекса в том, чтобы сравнивать методы, поэтому история запусков сохраняется. Сохранённые результаты можно сопоставлять — на совмещённых графиках и в таблице.

Наконец, комплекс применяется по назначению: проводятся вычислительные эксперименты, в которых реализованные методы сравниваются на тестовых функциях разной структуры и размерности, а результаты анализируются.

1 Исследовательская часть

1.1 Задача глобальной оптимизации и метаэвристики

Задача глобальной оптимизации состоит в поиске точки, в которой целевая функция принимает наименьшее значение:

$$f(x) \rightarrow \min, \quad x \in D \subset \mathbb{R}^d, \quad (1)$$

где f — целевая функция, D — допустимая область поиска, d — размерность пространства решений. Задача максимизации сводится к минимизации заменой f на $-f$, поэтому далее везде речь идёт о минимизации.

Сложность задачи определяется рельефом целевой функции. Если у функции единственный минимум, найти его несложно. Гораздо труднее функции с множеством локальных минимумов: наряду с глобальным минимумом у них есть много локальных, и отличить один от другого по поведению функции в окрестности точки невозможно. Классические градиентные методы движутся в сторону убывания функции и поэтому сходятся к ближайшему минимуму, не различая, локальный он или глобальный. Для таких задач они малопригодны. К тому же градиентные методы требуют, чтобы функция была дифференцируемой, а это условие выполняется не всегда.

Альтернативой служат метаэвристические алгоритмы. Это итеративные методы поиска, идеи которых заимствованы из природных процессов: эволюции, поведения стай, миграции видов. Их главная особенность — работа не с одной точкой, а с целой популяцией решений. За счёт этого алгоритм одновременно просматривает несколько областей пространства поиска, и вероятность застрять в локальном минимуме снижается. Разнообразие популяции поддерживается случайными механизмами: мутацией, обменом информацией между решениями, отбором. Метаэвристики не требуют дифференцируемости функции и не используют производные, поэтому подходят для широкого круга задач — в том числе для таких, где функция задана лишь алгоритмом её вычисления. Платят за это отсутствием гарантий: найденное решение может оказаться не глобальным минимумом, а лишь близким к нему. К этому классу относятся, в частности, генетические алгоритмы [4], метод роя частиц (Particle Swarm Optimization, далее — PSO) [5] и рассматриваемый в работе алгоритм биогеографической оптимизации.

Чтобы сравнивать алгоритмы между собой, используют наборы тестовых функций с заранее известными минимумами [1, 6]. Функции подбирают так, чтобы их рельеф был разным и проверял разные стороны поведения метода. В работе используются четыре такие функции.

Функция сферы — самая простая, с единственным минимумом:

$$f_{\text{sph}}(x) = \sum_{i=1}^d x_i^2. \quad (2)$$

Её глобальный минимум равен нулю и достигается в начале координат. На ней удобно оценивать скорость сходимости алгоритма, поскольку локальных ловушек нет. Поверхность и контурная карта функции сферы показаны на рисунке 1.

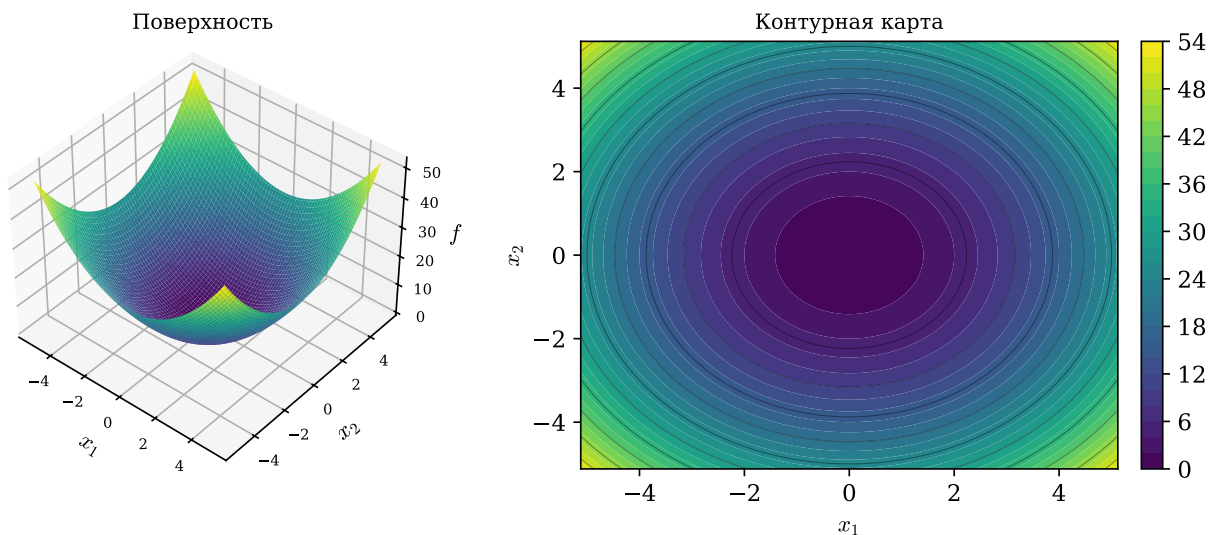


Рисунок 1 – Функция сферы: поверхность и контурная карта

У функции Растригина минимумов очень много:

$$f_{\text{ras}}(x) = Ad + \sum_{i=1}^d (x_i^2 - A \cos(2\pi x_i)), \quad A = 10. \quad (3)$$

Косинусное слагаемое создаёт большое число локальных минимумов, расположенных правильной решёткой. Глобальный минимум, равный нулю, находится в начале координат. Эта функция проверяет, способен ли алгоритм не застревать в локальных минимумах. Её рельеф приведён на рисунке 2.

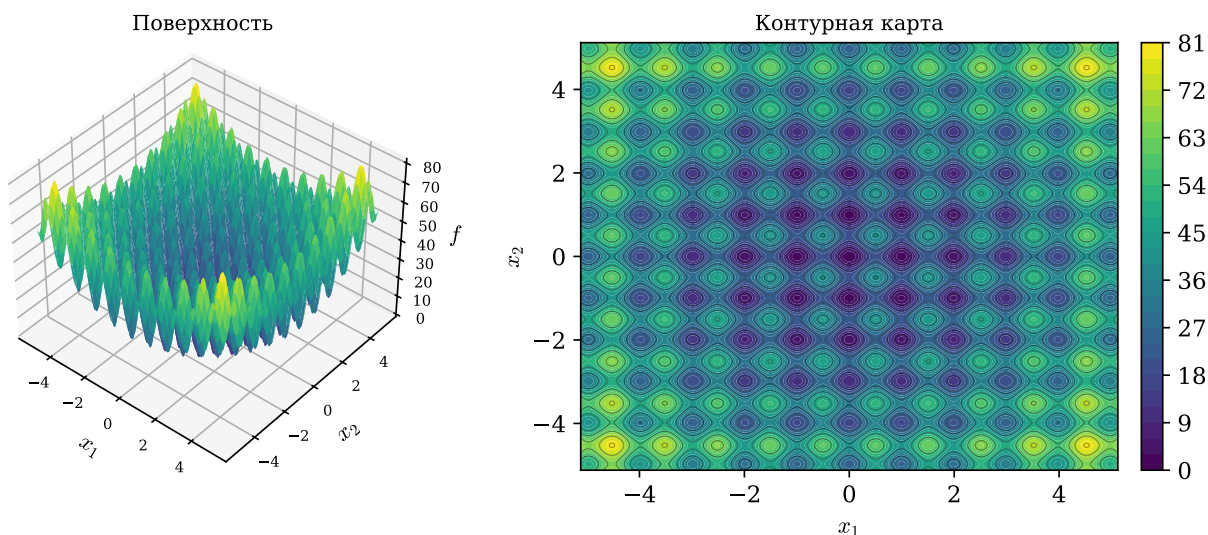


Рисунок 2 – Функция Растригина: поверхность и контурная карта

У функции Экли один глобальный минимум, окружённый почти плоской областью с мелкими неровностями:

$$f_{\text{ack}}(x) = -a \exp \left(-b \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \right) - \exp \left(\frac{1}{d} \sum_{i=1}^d \cos(c x_i) \right) + a + e, \quad (4)$$

где $a = 20$, $b = 0,2$, $c = 2\pi$. Минимум, равный нулю, достигается в начале координат. Вдали от минимума функция почти плоская и плохо подсказывает направление спуска, поэтому она проверяет, насколько точно метод выходит к минимуму. Поверхность и контурная карта функции Экли показаны на рисунке 3.

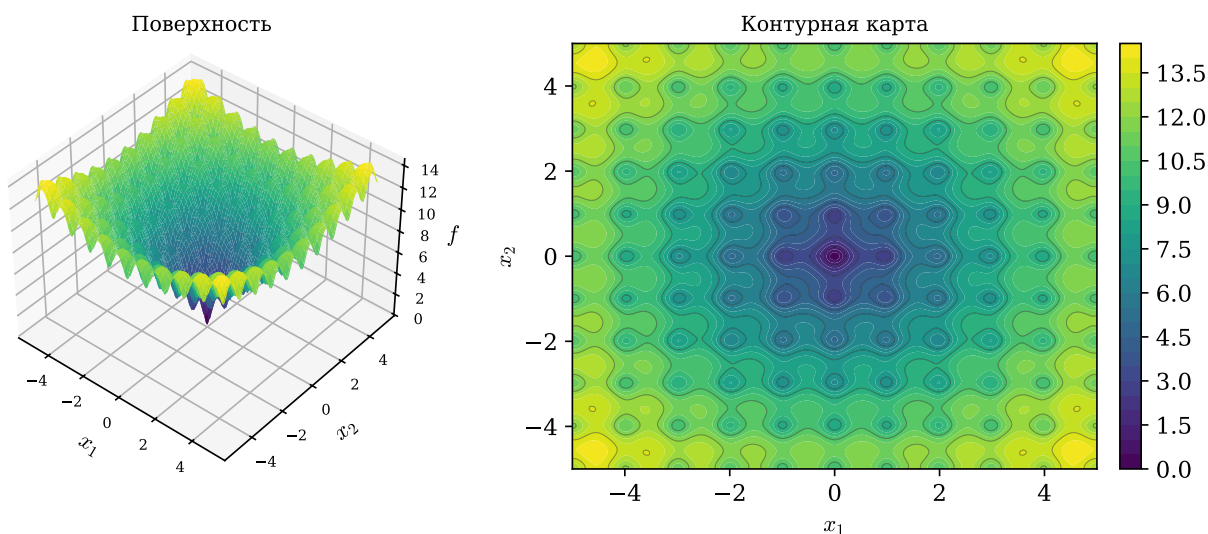


Рисунок 3 – Функция Экли: поверхность и контурная карта

Функция Букина N.6 определена только для двух переменных:

$$f_{\text{buk}}(x_1, x_2) = 100\sqrt{|x_2 - 0,01 x_1^2|} + 0,01 |x_1 + 10|. \quad (5)$$

Её минимум лежит вдоль узкого изогнутого оврага: малый шаг в сторону от него резко увеличивает значение функции. Эта функция проверяет, насколько хорошо алгоритм движется вдоль сложных по форме областей пространства решений. Её рельеф показан на рисунке 4.

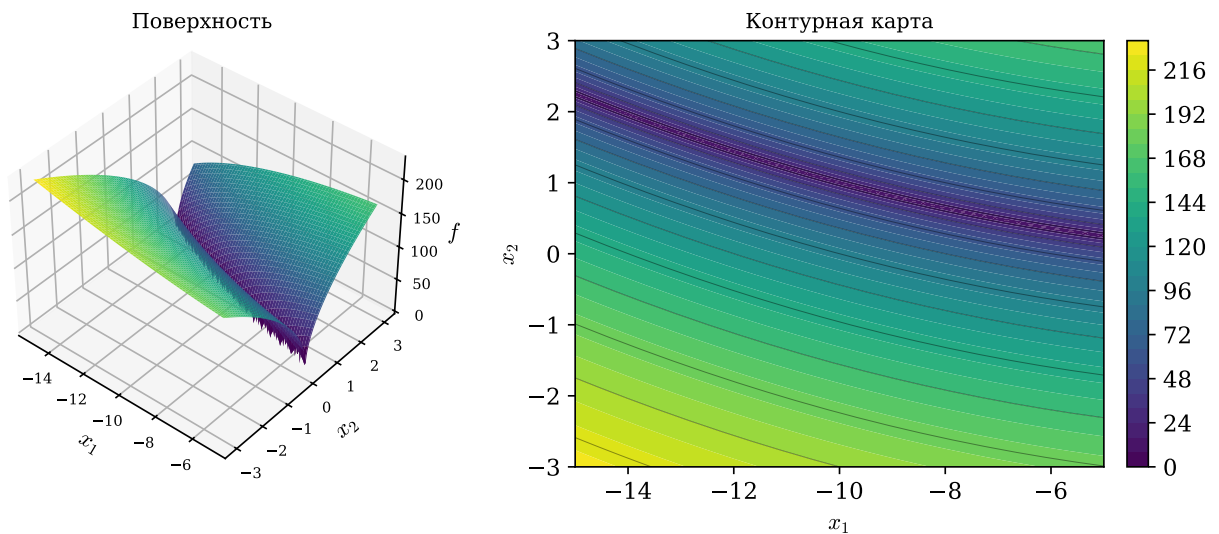


Рисунок 4 – Функция Букина N.6: поверхность и контурная карта

Такой набор охватывает задачи разного характера — от простой функции с единственным минимумом до функции с множеством локальных минимумов и до функции с узким оврагом, — что позволяет всесторонне оценить поведение метода.

1.2 Алгоритм биогеографической оптимизации

Алгоритм биогеографической оптимизации предложил Д. Саймон в 2008 году [2]. В его основе лежит модель биогеографии — науки о распределении живых организмов по островам. В терминах оптимизации каждое решение-кандидат считается отдельным островом, а качество решения — индексом пригодности среды обитания (Habitat Suitability Index, далее — HSI). Хорошему решению отвечает остров с высоким HSI, плохому — с низким. Хорошие острова охотно отдают свои признаки соседям, а плохие принимают чужие; так в популяции происходит обмен информацией [3].

У алгоритма два основных механизма — миграция и мутация. Пусть популяция состоит из N решений $X_1, \dots, X_N$, отсортированных по возрастанию значения целевой функции. Тогда ранг $r = 1$ получает худшее решение, а ранг $r = N$ — лучшее. Каждому решению сопоставляются две величины: скорость эмиграции λ_r , то есть готовность отдавать свои признаки, и скорость иммиграции μ_r — готовность принимать чужие. В линейной модели Саймона они задаются так:

$$\lambda_r = \frac{r}{N+1}, \quad \mu_r = 1 - \lambda_r, \quad r = 1, \dots, N. \quad (6)$$

Смысл формул простой: чем выше ранг решения, тем больше его скорость эмиграции и меньше скорость иммиграции. Лучшие решения становятся донорами признаков, худшие активнее принимают чужие. Вид линейных кривых миграции показан на левом графике рисунка 5.

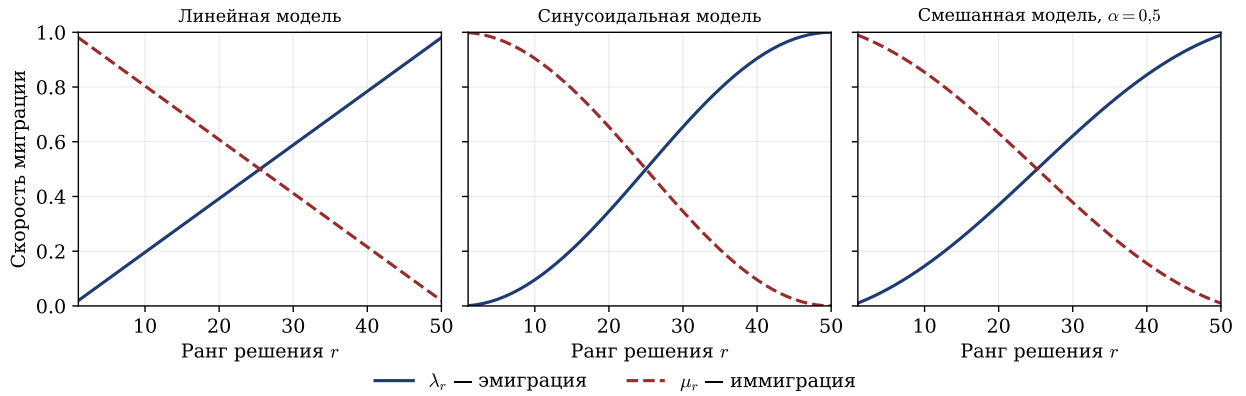


Рисунок 5 – Скорости эмиграции и иммиграции для линейной, синусоидальной и смешанной моделей миграции

Сам обмен идёт по схеме частичной иммиграции. Для каждого решения-получателя X_k , не входящего в число элитных, перебираются все его компоненты. Компонента с номером s заменяется с вероятностью μ_k : если замена происходит, на её место подставляется компонента s решения-донора. Донор выбирается случайно, причём вероятность стать донором пропорциональна скорости эмиграции λ_r , поэтому чаще донорами оказываются хорошие решения. Компоненты обновляются независимо друг от друга, и новое решение собирается из фрагментов нескольких разных доноров.

Мутация добавляет в популяцию случайность. Каждая компонента решения с вероятностью p_{mut} заменяется новым значением, равномерно выбран-

ным в допустимых границах. Это поддерживает разнообразие популяции и мешает ей преждевременно сойтись к одному минимуму.

Чтобы лучшие найденные решения не испортились при миграции и мутации, применяют элитарность: e лучших решений поколения переносятся в следующее без изменений. Один из распространённых способов сделать это таков: новые решения, полученные после миграции и мутации, оцениваются целевой функцией, после чего e худших из них заменяются сохранёнными элитными решениями.

Таким образом, одно поколение классического ВВО состоит из следующих шагов: вычислить целевую функцию для всей популяции и отсортировать решения по качеству; по рангу каждого решения найти λ_r и μ_r ; для всех неэлитных решений выполнить частичную иммиграцию, заменяя отдельные компоненты значениями доноров; применить мутацию; перенести элитные решения в новую популяцию; обновить лучшее найденное значение и историю сходимости. Эти шаги повторяются заданное число поколений.

1.3 Модификации модели миграции

В классическом ВВО скорости миграции линейно зависят от ранга решения. Форма кривой миграции определяет, насколько интенсивно лучшие решения передают свои признаки худшим и как обмен информацией распределён по популяции. Линейная модель распределяет эту интенсивность равномерно, и в ряде задач это приводит к преждевременной сходимости: решения быстро становятся похожими, а поиск теряет разнообразие [7]. Изменить поведение алгоритма можно, не трогая остальные его части, — достаточно заменить способ вычисления скоростей λ_r и μ_r .

Первый вариант — синусоидальная модель миграции. В ней кривые задаются так:

$$\lambda_r^{\sin} = \frac{1 - \cos\left(\pi \frac{r}{N}\right)}{2}, \quad \mu_r^{\sin} = 1 - \lambda_r^{\sin}, \quad r = 1, \dots, N. \quad (7)$$

Синусоидальная кривая нелинейна: на краях ранговой шкалы она пологая, а в середине меняется резко. Это видно на среднем графике рисунка 5. У группы лучших решений скорости эмиграции близки и велики, у группы худших так же близки и велики скорости иммиграции, а вблизи середины роль решения

быстро меняется с получателя на донора. В результате синусоидальная модель сильнее, чем линейная, разделяет популяцию на доноров и получателей признаков.

Второй вариант — смешанная модель. Её скорости строятся как взвешенная сумма линейной и синусоидальной кривых:

$$\lambda_r^{\text{mix}} = \alpha \lambda_r^{\text{lin}} + (1 - \alpha) \lambda_r^{\text{sin}}, \quad \mu_r^{\text{mix}} = 1 - \lambda_r^{\text{mix}}, \quad r = 1, \dots, N, \quad (8)$$

где $\alpha \in [0, 1]$ — параметр смешения, задающий долю линейной кривой. При $\alpha = 1$ модель совпадает с линейной, при $\alpha = 0$ — с синусоидальной, а промежуточные значения дают форму кривой между ними. Это позволяет настраивать баланс между широким исследованием пространства и уточнением уже найденных решений. Смешанная кривая при $\alpha = 0,5$ показана на правом графике рисунка 5.

Все три модели — линейная, синусоидальная и смешанная — используют одну и ту же схему иммиграции и мутации. Различаются они только формулами для λ_r и μ_r . Благодаря этому модель миграции остаётся самостоятельным и легко заменяемым компонентом алгоритма: переход от одного варианта к другому сводится к замене способа расчёта скоростей, а вся остальная схема поиска не меняется.

1.4 Модификации базового алгоритма

Кроме изменения формы кривых миграции, поиск можно улучшить, дополнив базовый алгоритм механизмами адаптации. Все три механизма, описанные ниже, не зависят от модели миграции и применимы к любому варианту ВВО.

Первый механизм — адаптивная мутация. В базовом алгоритме вероятность мутации p_{mut} постоянна на протяжении всей работы. Однако в начале поиска высокая мутация полезна, поскольку поддерживает разнообразие популяции и помогает охватить широкую область пространства решений, а ближе к концу она лишь вносит шум и мешает уточнять найденные минимумы. Поэтому вероятность мутации делают убывающей: она линейно снижается от начального значения P_{start} до конечного P_{end} по мере смены поколений. В

поколении t её значение равно

$$p_{\text{mut}}^{(t)} = P_{\text{start}} + \frac{t}{T-1}(P_{\text{end}} - P_{\text{start}}), \quad t = 0, 1, \dots, T-1, \quad (9)$$

где T — общее число поколений. Ранние поколения работают с высокой мутацией, поздние — с пониженной [1].

Второй механизм — детектор стагнации и частичный рестарт. Стагнацией называют ситуацию, когда лучшее найденное значение целевой функции не улучшается несколько поколений подряд. Базовый ВВО не умеет из неё выходить. Детектор стагнации подсчитывает поколения без улучшений; как только счётчик достигает порога W , выполняется частичный рестарт: доля ρ худших решений удаляется и заменяется новыми случайными решениями из допустимой области. Одновременно текущая вероятность мутации временно увеличивается в k раз, но не выше P_{start} . После рестарта счётчик обнуляется, а линейное убывание мутации продолжается [8]. Такой механизм помогает выбраться из локального минимума, не теряя при этом уже найденные хорошие решения.

Третий механизм — локальный поиск. Популяционные алгоритмы хорошо исследуют пространство в целом, но часто недостаточно точны при уточнении минимума. Чтобы это исправить, через заданное число поколений выбирается несколько лучших решений, и каждое из них уточняется серией шагов. На каждом шаге к решению добавляется случайное гауссовское возмущение с нулевым средним и масштабом, пропорциональным ширине допустимой области. Масштаб возмущения убывает с ростом номера поколения: в начале шаги крупнее, в конце мельче. Если возмущённая точка оказывается лучше, она принимается, иначе отвергается. Локальный поиск применяют только к лучшим решениям, чтобы не тратить вычисления на заведомо неперспективные [1].

Три механизма дополняют друг друга: адаптивная мутация управляет уровнем случайности на всём протяжении поиска, детектор стагнации обеспечивает выход из локальных ловушек, а локальный поиск повышает точность итогового результата. Они независимы друг от друга, поэтому могут применяться как вместе, так и по отдельности.

1.5 Метод роя частиц и сравнение с биогеографической оптимизацией

Метод роя частиц предложили Дж. Кеннеди и Р. Эберхарт в 1995 году [5]. Его идея навеяна поведением стаи птиц или косяка рыб при поиске пищи. Популяция здесь — это рой частиц, и каждая частица соответствует одному решению. Состояние частицы i задаётся положением $x_i \in \mathbb{R}^d$ и скоростью $v_i \in \mathbb{R}^d$.

На каждой итерации t скорость и положение частицы пересчитываются по формулам

$$v_i^{(t+1)} = \omega v_i^{(t)} + c_1 r_1 \left(p_{i,\text{best}}^{(t)} - x_i^{(t)} \right) + c_2 r_2 \left(g_{\text{best}}^{(t)} - x_i^{(t)} \right), \quad (10)$$

$$x_i^{(t+1)} = x_i^{(t)} + v_i^{(t+1)}, \quad (11)$$

где $p_{i,\text{best}}$ — лучшее положение, в котором частица i побывала за всё время; g_{best} — лучшее положение среди всех частиц роя; ω — инерционный коэффициент; c_1 и c_2 — когнитивная и социальная составляющие; $r_1, r_2 \sim U(0,1)$ — случайные числа [9].

Инерционный коэффициент ω задаёт баланс между исследованием пространства и уточнением найденного. При большом ω частицы сохраняют направление движения и шире обследуют область поиска, при малом — сильнее притягиваются к лучшим найденным точкам. На практике ω часто линейно убывает в ходе работы — от значения около 0,9 в начале до 0,4 в конце. Чтобы частицы не разлетались слишком далеко за одну итерацию, скорость по каждой координате обычно ограничивают по модулю некоторой величиной v_{max} , а вышедшее за границы положение возвращают в допустимую область [10].

Несмотря на внешнее сходство — оба алгоритма работают с популяцией решений и обмениваются информацией внутри неё — механизмы этого обмена устроены по-разному.

В PSO каждая частица учитывает два ориентира: своё лучшее прошлое положение и глобально лучшую точку роя. На задачах с простым рельефом это даёт быструю сходимость — рой быстро стягивается к минимуму. Но на задачах с множеством локальных минимумов тот же механизм оборачивается слабостью: если глобально лучшая точка попала в окрестность локального минимума, рой собирается там, и выбраться оттуда без перезапуска трудно.

В BBO обмен идёт покомпонентно: каждая компонента решения может быть независимо заимствована у любого донора. Это создаёт более разнообразный поток информации и мешает популяции полностью унифицироваться. Вдобавок элитарность защищает лучшие решения, а мутация постоянно вносит случайные возмущения. В сумме BBO устойчивее на сложных задачах высокой размерности, хотя на простых функциях сходится несколько медленнее [3].

Поэтому PSO и BBO занимают разные ниши. PSO удобен там, где рельеф простой и важна скорость сходимости. BBO и его модификации выигрывают на задачах с множеством локальных минимумов, где важно сохранять разнообразие популяции на протяжении всего поиска. Это согласуется с общим положением о том, что эффективность метода оптимизации зависит от свойств целевой функции и ни один метод не превосходит остальные на всех задачах [11]. В работе PSO используется как метод сравнения, относительно которого оценивается поведение вариантов BBO.

1.6 Методы визуализации результатов оптимизации

Поведение метаэвристик трудно оценить по одним лишь числам, поэтому результаты их работы принято представлять наглядно. Визуализация решает несколько задач: она позволяет наблюдать за ходом поиска и замечать нежелательные явления — преждевременную сходимость, стагнацию, потерю разнообразия популяции, — а также сравнивать алгоритмы между собой и понимать, как метод исследует пространство решений [1]. Сложились несколько типов визуального представления, каждый из которых решает свою задачу.

Наиболее универсальное средство — график сходимости. По оси абсцисс на нём откладывается номер поколения, по оси ординат — лучшее значение целевой функции, найденное к этому поколению:

$$f_{\text{best}}^{(t)} = \min_{0 \leq \tau \leq t} f\left(x_{\text{best}}^{(\tau)}\right). \quad (12)$$

По построению такой график монотонно невозрастающий: лучшее найденное значение не может ухудшиться. По форме кривой судят о характере сходимости. Резкий начальный спад с выходом на плато говорит о быстрой сходимости к некоторому значению, которое может оказаться локальным минимумом. Плавный равномерный спад свидетельствует о планомерном ис-

следовании пространства. Ступенчатая кривая характерна для алгоритмов с рестартом: после каждого рестарта метод находит лучшее решение, и кривая делает скачок вниз. Когда значения целевой функции убывают на несколько порядков, по оси ординат применяют логарифмическую шкалу — иначе весь интересный участок кривой сжимается у нуля и становится нечитаемым. Для сравнения методов кривые их сходимости совмещают на одних осях, различая цветом и типом линии.

График сходимости показывает поведение алгоритма в терминах значений функции, но не говорит, где именно в пространстве решений находится популяция. Это показывает визуализация пространства поиска. При размерности $d = 2$ строится контурная карта: по осям откладываются координаты x_1 и x_2 , а значение $f(x_1, x_2)$ кодируется цветом, причём тёмные тона отвечают малым значениям функции, то есть перспективным областям [12]. Допустимая область разбивается на равномерную сетку, для каждой узловой точки вычисляется значение функции, и полученная матрица отображается как изображение; поверх него наносятся маркеры особей популяции. Если сетка содержит $M \times M$ точек, для построения карты требуется M^2 вычислений функции, поэтому карту рассчитывают один раз до запуска, а в ходе работы обновляют только маркеры. При $d > 2$ прямое отображение невозможно: тогда используют первые две координаты особей, а остальные фиксируют, строя сечение функции. Такой подход даёт лишь приближённое представление, но позволяет судить о расположении популяции и в многомерных задачах.

Контурная карта кодирует значение функции цветом, но не передаёт объёмной структуры рельефа: глубину минимумов и крутизну склонов по оттенку оценить трудно. Эту структуру передаёт трёхмерный поверхностный график, где значение $f(x_1, x_2)$ отображается высотой точки. Поверхность строится по той же сетке, что и контурная карта, а соседние узлы образуют четырёхугольные грани. Чтобы показать трёхмерную сцену на плоском экране, применяют проекцию, а грани рисуют по алгоритму художника — от дальних к ближним, так что ближние перекрывают дальние без использования буфера глубины.

Главное преимущество трёхмерного графика — возможность вращения. Точка обзора задаётся двумя углами: поворотом вокруг вертикальной оси φ_z и вокруг горизонтальной оси φ_x . При изменении углов координаты

каждой вершины (x, y, z) пересчитываются последовательным поворотом в двух плоскостях:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \varphi_z & -\sin \varphi_z \\ \sin \varphi_z & \cos \varphi_z \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}, \quad (13)$$

$$\begin{pmatrix} y'' \\ z'' \end{pmatrix} = \begin{pmatrix} \cos \varphi_x & -\sin \varphi_x \\ \sin \varphi_x & \cos \varphi_x \end{pmatrix} \begin{pmatrix} y' \\ z \end{pmatrix}. \quad (14)$$

Экранными координатами вершины служат x' и y'' с учётом масштаба и смещения к центру холста. Вращение позволяет рассмотреть поверхность под любым углом и заметить узкие овраги целевой функции, плохо различимые на плоской карте. Поверх поверхности можно нанести траекторию лучшего решения — последовательность точек $(x_1^{(t)}, x_2^{(t)}, f_{\text{best}}^{(t)})$, соединённых линией, которая показывает, как метод продвигался по рельефу.

Контурная карта и поверхность дают снимок одного состояния, но для понимания поиска важна динамика — как популяция смещалась от поколения к поколению. Зафиксировать её позволяет анимация в формате обмена графическими данными (Graphics Interchange Format, далее — GIF), который хранит последовательность кадров, сменяющих друг друга с заданной частотой [13]. Каждый кадр — это контурная карта с маркерами особей на соответствующем поколении; накопленные кадры собираются в один файл. GIF удобен тем, что не требует внешних плееров, просто собирается из готовых изображений и легко сохраняется. Его ограничение — не более 256 цветов в кадре и заметный размер файла при большом числе кадров, но для задач визуализации оптимизации это не критично.

1.7 Обзор программ-аналогов

Задача наблюдения за работой алгоритмов оптимизации не нова, и для неё уже создан ряд программных средств. Чтобы понять, какое место занимает разрабатываемая система, рассмотрим несколько существующих инструментов и отметим их сильные и слабые стороны.

Инструмент STNWeb — это веб-приложение для анализа поведения метаэвристик. Он строит так называемые сети траекторий поиска: ориентированные графы, по которым видно, как алгоритм перемещается по пространству решений. STNWeb позволяет сравнивать несколько прогонов нескольких

алгоритмов на одной задаче и помогает понять, почему тот или иной метод работает хуже [14]. Сильная сторона инструмента — глубокий анализ траекторий и удобное сравнение алгоритмов. Однако STNWeb ориентирован на исследователей, требует подготовки входных данных в специальном формате и не показывает ход поиска в реальном времени: он анализирует уже завершённые запуски.

Проект OptimViz решает другую задачу — он наглядно показывает, как различные методы оптимизации шаг за шагом движутся по поверхности тестовой функции, и сохраняет результат в виде анимаций [15]. Это удобно для обучения и демонстраций. Слабые стороны: инструмент написан на языке MATLAB и требует платных дополнительных пакетов, а сами анимации создаются заранее, без возможности интерактивно менять параметры запуска.

Веб-демонстрация Э. Дюпона позволяет прямо в браузере выбрать точку старта на контурной карте двумерной функции и посмотреть, как к минимуму сходятся разные алгоритмы [16]. Она интерактивна и наглядна, но ограничена двумя измерениями и фиксированным набором функций, а сами алгоритмы относятся к градиентным методам обучения нейронных сетей, а не к метаэвристикам.

Дашборд moPLOT строит детальные изображения рельефа тестовых функций, в том числе для задач многокритериальной оптимизации [17]. Он даёт качественную визуализацию ландшафта, но предназначен в первую очередь для анализа самих функций, а не для наблюдения за работой конкретного алгоритма, и требует навыков работы со средой R.

Сравнение показывает, что существующие инструменты решают близкие задачи, но каждый со своими ограничениями. Одни анализируют только завершённые запуски, другие работают лишь с двумя измерениями или с заранее подготовленными анимациями, третьи требуют специальной среды и рассчитаны на опытных исследователей. Кроме того, все рассмотренные средства предназначены для настольных компьютеров или браузера и не приспособлены для работы на мобильном устройстве.

Разрабатываемая система занимает свободную нишу. Она сочетает наблюдение за ходом поиска в реальном времени, визуализацию пространства решений в виде контурной карты и трёхмерной поверхности, анимацию динамики популяции и сравнение нескольких запусков, причём всё это доступно

на мобильном устройстве и не требует от пользователя настройки вычислительного окружения. Основное внимание уделено семейству алгоритмов биогеографической оптимизации и их модификациям, что делает систему удобным средством для изучения именно этих методов.

2 Конструкторская часть

2.1 Функциональные требования к программному комплексу

Прежде чем переходить к проектированию, нужно определить, что комплекс должен делать. Требования удобно разделить на функциональные, описывающие возможности системы с точки зрения пользователя, и нефункциональные, задающие общие свойства системы.

Основной сценарий работы — постановка и проведение вычислительного эксперимента. Пользователь должен иметь возможность выбрать метод оптимизации из доступного семейства, задать целевую функцию и параметры задачи, после чего запустить вычисление. В качестве целевой функции система должна принимать как одну из встроенных тестовых функций, так и произвольное выражение, введённое пользователем. Для задачи пользователь должен задавать размерность и границы области поиска, а для выбранного метода — его параметры, например размер популяции, число поколений и вероятность мутации.

Во время вычисления система должна показывать ход поиска, не дожидаясь его завершения. Пользователю нужно видеть номер текущего поколения, лучшее найденное значение целевой функции и график сходимости, который обновляется по мере поступления новых данных. Кроме того, должна быть возможность отменить запущенную задачу и видеть положение задачи в очереди, если она ещё не начала выполняться.

После завершения эксперимента система должна представить его результат: лучшее найденное решение и значение функции в нём, график сходимости за весь запуск, а также числовые показатели качества. Для наглядного представления пространства поиска результат должен сопровождаться визуализацией: контурной картой целевой функции с нанесёнными на неё решениями, трёхмерной поверхностью функции с возможностью её вращения и анимацией, показывающей, как менялась популяция от поколения к поколению.

Способы визуализации, рассмотренные в подразделе 1.6, накладывают на систему ряд требований. Прежде всего график сходимости должен обновляться инкрементально: новые точки добавляются к уже построенной кривой по мере поступления данных, а не строятся каждый раз заново. Это

необходимо, чтобы пользователь видел ход поиска во время вычислений, и одновременно снижает нагрузку на устройство. Затем, поскольку экран смартфона невелик, представление данных должно быть лаконичным: с читаемыми подписями осей и легендой, не перекрывающей основной график. Наконец, построение контурной карты и трёхмерной поверхности требует многократного вычисления целевой функции, поэтому карта и поверхность должны рассчитываться один раз, а при наблюдении за поиском и вращении должны обновляться только положения маркеров и точка обзора, но не сам рельеф.

Результаты экспериментов должны сохраняться, чтобы пользователь мог вернуться к ним позже. Система должна хранить историю проведённых запусков и позволять просматривать её, отбирать отдельные запуски и сравнивать несколько результатов между собой — на совмещённом графике сходимости, в виде таблицы показателей и столбчатой диаграммы.

Нефункциональные требования определяются назначением системы и условиями её использования. Поскольку основным устройством пользователя является смартфон, приложение должно работать на мобильной платформе и оставаться отзывчивым: интерфейс не должен подвисать во время вычислений и построения графиков. Сами вычисления могут быть длительными и ресурсоёмкими, поэтому их следует выполнять отдельно от устройства пользователя, на сервере. Сервер, в свою очередь, должен поддерживать одновременную работу нескольких клиентов и упорядочивать поступающие от них задачи.

2.2 Архитектура программного комплекса

Перечисленные возможности естественно распределяются между двумя разными по назначению частями системы. Одна обращена к пользователю и должна работать на смартфоне, другая выполняет ресурсоёмкие вычисления. Совмещать их в одном приложении нецелесообразно: оптимизация на больших популяциях и высоких размерностях нагружает процессор, а мобильное устройство ограничено по вычислительной мощности и расходу батареи. Поэтому выбрана клиент-серверная архитектура, в которой вычисления вынесены на сервер, а мобильное приложение отвечает за управление и отображение результатов [18]. Общая структура комплекса показана на рисунке 6.

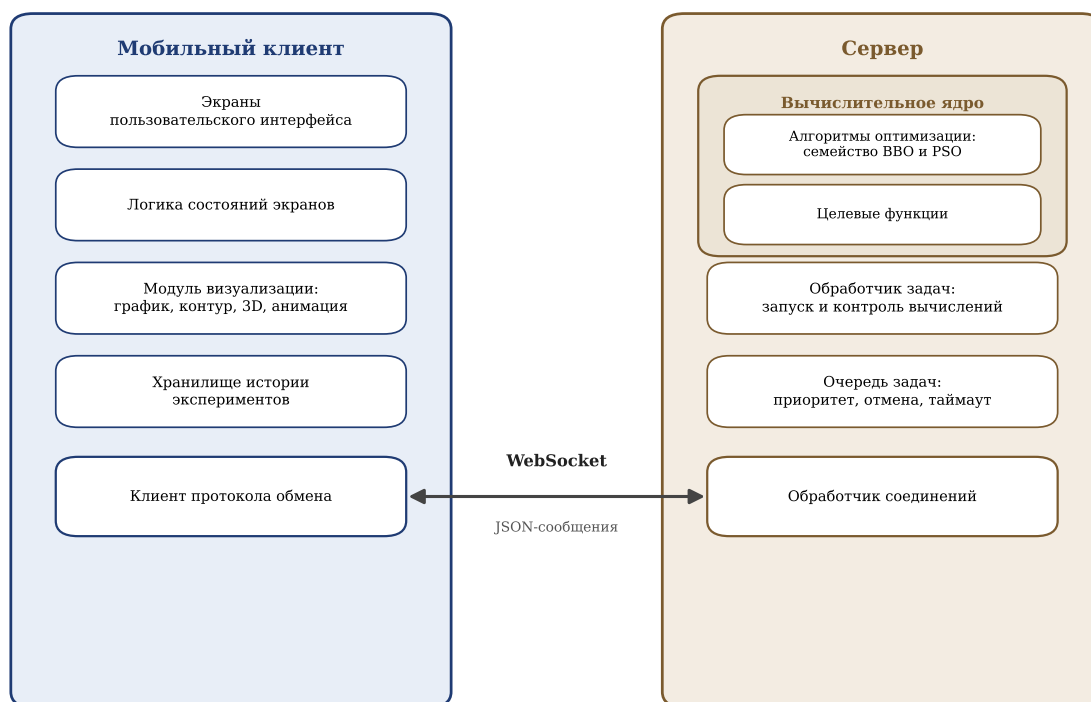


Рисунок 6 – Архитектура программного комплекса

Мобильный клиент — это приложение, с которым непосредственно работает пользователь. Оно состоит из нескольких слоёв. Экраны образуют пользовательский интерфейс: настройку эксперимента, наблюдение за прогрессом, просмотр результата и работу с историей. За экранами стоит слой логики состояний, который хранит текущее состояние каждого экрана и реагирует на действия пользователя и на данные, приходящие с сервера. Отдельный модуль отвечает за визуализацию — построение графика сходимости, контурной карты, трёхмерной поверхности и анимации. История проведённых экспериментов хранится в локальной базе данных на самом устройстве, что позволяет работать с прошлыми результатами и без подключения к серверу. Наконец, клиент протокола обмена обеспечивает связь с сервером.

Сервер выполняет вычисления и не имеет собственного интерфейса. Все запросы клиентов принимает обработчик соединений: он разбирает входящие сообщения и направляет их дальше. Поскольку клиентов может быть несколько, а вычисления длительны, поступающие задачи помещаются в очередь, которая упорядочивает их и позволяет отменять ещё не выполненные. Выполнением занимается обработчик задач: он берёт задачу из очереди, запускает соответствующий алгоритм и по ходу работы отправляет клиенту

данные о прогрессе. Сами алгоритмы оптимизации — семейство ВВО и метод роя частиц — вместе с целевыми функциями образуют вычислительное ядро сервера. Какой именно алгоритм запускать и какую функцию вычислять, обработчик задач определяет по сведениям из поступившего запроса.

Взаимодействие между клиентом и сервером построено поверх протокола WebSocket. В отличие от обычного запроса и ответа по протоколу HTTP, WebSocket держит постоянное двустороннее соединение, по которому сервер может в любой момент присылать клиенту новые данные [19, 20]. Это и нужно для передачи прогресса: пока идёт вычисление, сервер по тому же соединению отправляет клиенту сведения о каждом поколении, и приложение обновляет график сходимости в реальном времени. Сообщения передаются в текстовом формате JSON, общая структура которого описана в следующем подразделе.

2.3 Проектирование протокола обмена данными

Клиент и сервер обмениваются сообщениями по постоянному соединению WebSocket. Чтобы этот обмен был упорядоченным и расширяемым, нужен прикладной протокол — набор правил о том, какие сообщения бывают, как они устроены и в каком порядке ими обмениваются. Все сообщения протокола передаются в текстовом формате JSON [21]. Этот формат выбран потому, что он удобочитаем, поддерживается без дополнительных библиотек в типовых веб-приложениях, и его разбор встроен в стандартные средства как серверного, так и клиентского языка [22].

Каждое сообщение имеет единую структуру и состоит из двух частей — заголовка и полезной нагрузки. Заголовок содержит служебные поля, общие для всех сообщений: версию протокола, тип сообщения, его уникальный идентификатор, метку времени, а также поля для связи сообщений между собой и для идентификации задачи. Полезная нагрузка зависит от типа сообщения и содержит его данные: например, параметры запускаемой задачи или сведения об очередном поколении. Такое разделение позволяет обрабатывать служебную часть единообразно, не разбирая каждый тип сообщения по-своему.

Все сообщения делятся на несколько групп по назначению. Служебные сообщения обслуживают само соединение:

- `hello` — рукопожатие при подключении и обмен сведениями о возможностях сторон;
- `ping` и `pong` — проверка того, что соединение живо;
- `error` — сообщение об ошибке протокола, проверки данных или выполнения.

Сообщения управления задачами обеспечивают её постановку и сопровождение:

- `job.submit` — запрос на запуск задачи с указанием метода, целевой функции и параметров;
- `job.accepted` — подтверждение приёма задачи и присвоение ей идентификатора;
- `cancel` — запрос на отмену задачи;
- `job.status.get` и `job.status` — запрос текущего состояния задачи и ответ на него.

Сообщения о ходе и итогах вычисления идут от сервера к клиенту:

- `job.started` — уведомление о начале выполнения задачи;
- `job.progress` — данные об очередном поколении: его номер и лучшее найденное значение;
- `job.result` — итоговый результат с лучшим решением и показателями качества;
- `job.finished` — уведомление о завершении задачи с указанием итогового состояния.

Порядок, в котором сервер меняет состояние задачи и шлёт эти сообщения, образует жизненный цикл задачи. Он показан на рисунке 7. Поступившая задача сначала попадает в очередь. Когда обработчик задач берёт её в работу, она переходит в состояние выполнения, и клиенту отправляется уведомление о начале. Во время выполнения сервер периодически шлёт сообщения о прогрессе. Завершиться задача может по-разному: успешно, с ошибкой, по отмене пользователем или по превышению отведённого времени. Отменить задачу можно и пока она ещё стоит в очереди.

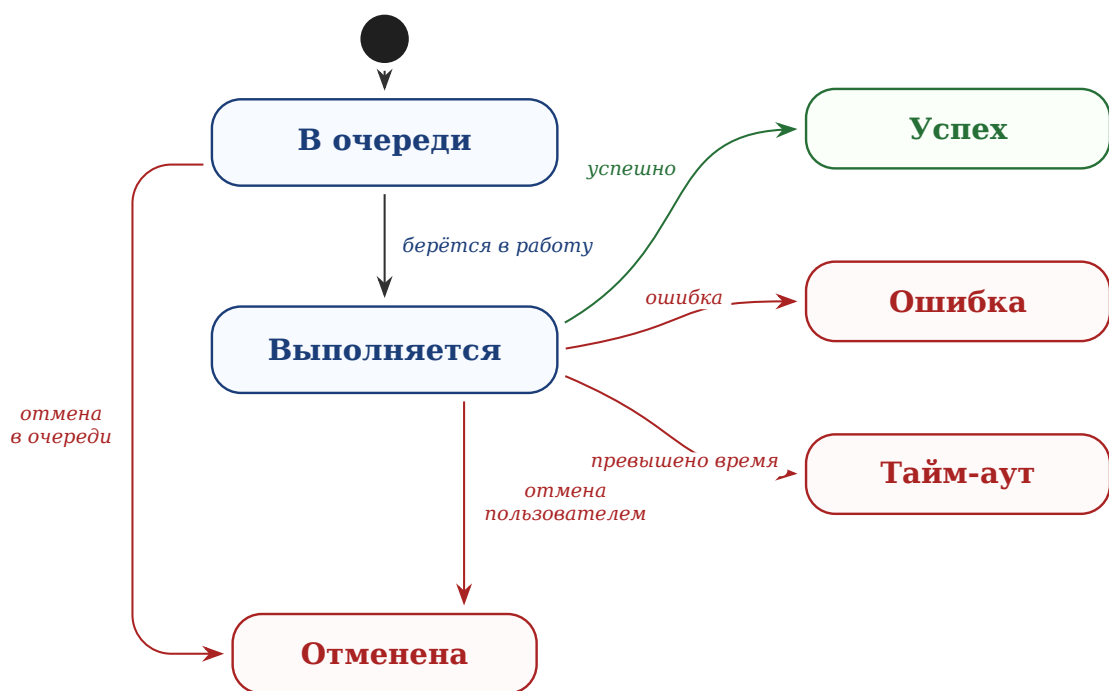


Рисунок 7 – Жизненный цикл задачи на сервере

В протоколе предусмотрены и средства для передачи больших объёмов данных. Если сообщение превышает допустимый размер, его полезную нагрузку можно разбить на части — чанки — и собрать на стороне клиента по общему идентификатору потока. Аналогично заголовок предусматривает возможность сжатия полезной нагрузки. Эти средства заложены в протокол на будущее: при текущих объёмах данных, которыми обмениваются клиент и сервер, разбиение на части и сжатие не требуются, и сообщения передаются целиком в несжатом виде.

2.4 Модель хранения данных

Одно из требований к системе — сохранять историю проведённых экспериментов, чтобы пользователь мог возвращаться к прошлым запускам и сравнивать их между собой. Отсюда вытекает несколько задач, которые должно решать хранилище. Во-первых, по каждому запуску нужно сохранять достаточно сведений, чтобы его можно было однозначно опознать в списке и понять, что именно вычислялось: каким методом, на какой задаче и с каким итогом. Во-вторых, нужно хранить сам результат запуска — не только итоговое число, но и данные, по которым строятся графики и визуализация, иначе к завершённому эксперименту нельзя будет вернуться позже. В-третьих, за-

писи должны быть однородными, чтобы их можно было перебирать, отбирать по условию и сопоставлять друг с другом.

Чтобы учесть эти требования, прежде всего нужно понять, какие данные и в каких отношениях между собой следует хранить. Для этого построена концептуальная модель данных в виде диаграммы «сущность-связь», показанная на рисунке 8. Она описывает предметную область на уровне сущностей и связей, без привязки к конкретному способу хранения.

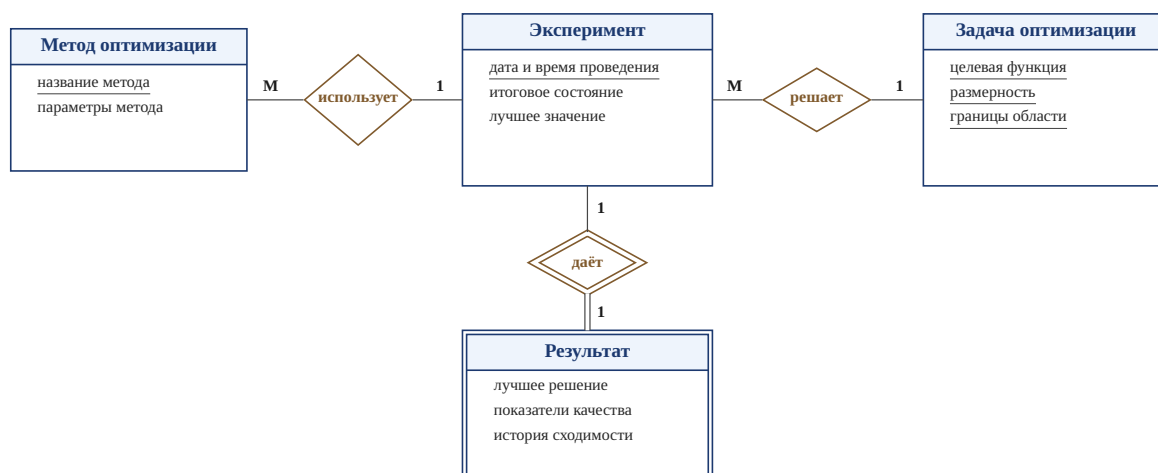


Рисунок 8 – Модель «сущность-связь» истории экспериментов

Центральная сущность модели — эксперимент. Он соответствует одному завершённому запуску оптимизации и описывается датой и временем проведения, итоговым состоянием и лучшим найденным значением целевой функции. Ключом служит дата и время проведения: запуск однозначно определяется моментом, в который он был начат.

С экспериментом связаны три другие сущности. Метод оптимизации описывает, каким алгоритмом проводился запуск, и хранит название метода и значения его параметров. Ключом этой сущности является название метода, которое однозначно его определяет. Один и тот же метод может использоваться во многих экспериментах, поэтому связь «использует» имеет тип «многие к одному». Задача оптимизации описывает, что именно решалось, и состоит из целевой функции, размерности и границ области поиска. Эти три атрибута вместе образуют составной ключ: задача определяется их сочетанием, поскольку одна и та же функция в разных размерностях задаёт разные задачи. Одна и та же задача также может встречаться в разных экспериментах, поэтому связь «решает» тоже имеет тип «многие к одному».

Результат хранит итог запуска: лучшее найденное решение, числовые показатели качества и историю сходимости, по которой строится график. Эта сущность не имеет собственного ключа и не существует отдельно от эксперимента, поэтому она моделируется как слабая сущность, идентифицируемая через эксперимент. Связь «даёт», соединяющая их, является идентифицирующей и имеет тип «один к одному»: каждому эксперименту соответствует ровно один результат.

Такая модель отражает естественную структуру предметной области: эксперимент объединяет вокруг себя сведения о применённом методе, решаемой задаче и полученном результате. История экспериментов представляет собой набор однородных записей такого вида, который пополняется после каждого завершённого запуска и хранится на устройстве пользователя.

2.5 Проектирование пользовательского интерфейса

Интерфейс приложения строится вокруг основного сценария работы: настроить эксперимент, запустить его, понаблюдать за ходом вычислений и изучить результат. Чтобы каждый шаг этого сценария был понятен и не перегружал пользователя, он вынесен на отдельный экран. Кроме того, нужны экраны для работы с историей и сравнения запусков. Связи между экранами и переходы между ними показаны на рисунке 9.



Рисунок 9 – Схема навигации между экранами приложения

Работа начинается с экрана подключения, на котором задаётся адрес сервера и устанавливается соединение. После успешного подключения пользователь попадает на экран настройки — центральный экран приложения. Здесь выбирается метод оптимизации, задаётся целевая функция, размерность и границы области поиска, а также параметры выбранного метода. С этого же экрана можно перейти к истории прошлых экспериментов.

После запуска эксперимента открывается экран прогресса. На нём отображается номер текущего поколения, лучшее найденное значение и график сходимости, который обновляется в реальном времени по мере поступления данных с сервера. Когда вычисление завершается, приложение переходит на экран результата.

Экран результата показывает итог запуска: лучшее найденное решение, числовые показатели и график сходимости за весь запуск, а также визуализацию пространства поиска в виде контурной карты. С него можно открыть отдельный экран с трёхмерной поверхностью целевой функции, вернуться к настройке для нового запуска или перейти к истории. Завершённый эксперимент при этом сохраняется в историю.

Экран истории содержит список ранее проведённых экспериментов. Любую запись из него можно открыть, чтобы снова увидеть её результат, а отметив несколько записей, перейти к их сравнению. На экране сравнения результаты нескольких запусков выводятся вместе: на совмещённом графике сходимости, в виде таблицы показателей и столбчатой диаграммы. Такое разделение на экраны даёт простой и предсказуемый путь от постановки задачи до анализа и сравнения результатов.

3 Технологическая часть

3.1 Обоснование выбора технологий

Архитектура, описанная в конструкторской части, не привязана к конкретным технологиям. В этом подразделе обосновывается выбор средств, которыми комплекс реализован: транспортного протокола, серверного и клиентского языков, модели параллельной обработки и формата обмена данными.

Транспортный протокол должен удовлетворять трём условиям. Сервер обязан отправлять данные клиенту по собственной инициативе, после каждого поколения и не дожидаясь запроса. Соединение должно оставаться открытым всё время работы задачи. Накладные расходы на сообщение должны быть малыми, поскольку сообщения о прогрессе идут потоком — по одному на поколение. Этим условиям отвечают не все распространённые решения. Классический обмен по протоколу HTTP с периодическим опросом сервера прост, но неэффективен: большинство запросов возвращают пустой ответ, а интервал опроса задаёт минимальную задержку между появлением данных и их получением [18]. Технология однонаправленной потоковой передачи событий от сервера снимает проблему опроса, но не поддерживает обратную связь: клиент не может по тому же соединению отправить задачу или команду отмены. Всем трём условиям отвечает протокол WebSocket. Он устанавливает постоянное двустороннее соединение поверх TCP, по которому обе стороны шлют сообщения в любой момент, а заголовок каждого кадра занимает лишь несколько байт [19]. Поэтому в качестве транспорта выбран WebSocket.

Серверная часть написана на языке Python. Он стал стандартным языком научных вычислений во многом благодаря библиотеке NumPy [23], которая обеспечивает быстрые операции над многомерными массивами [24]. Операции над всей популяцией — вычисление целевой функции для всех особей, сортировка, отбор доноров — выражаются через действия над массивами NumPy и выполняются на уровне скомпилированного кода, что значительно быстрее эквивалентных циклов на чистом Python.

Сервер должен обслуживать несколько клиентов одновременно, не останавливая работу с одним из них на время вычислений для другого. Для этого применяется асинхронная модель Python на основе библиотеки `asyncio`: мно-

жество соединений обслуживаются в одном потоке за счёт того, что операция, ожидающая данных по сети, уступает управление другим [24]. Однако сам алгоритм оптимизации вычислительно нагружен и не имеет точек ожидания, поэтому при запуске прямо в цикле событий он заблокировал бы все остальные соединения. Чтобы этого избежать, алгоритм запускается в отдельном потоке из пула потоков, а данные о прогрессе передаются из него обратно в цикл событий через потокобезопасную очередь. Веб-часть сервера построена на фреймворке FastAPI, который поддерживает соединения WebSocket и работает поверх той же асинхронной модели [25].

Сообщения протокола сериализуются в формате JSON. Этот формат поддерживается без дополнительных библиотек и в Python, и в Dart [26, 27], а его текстовое представление удобно читать при отладке. Существуют и более компактные двоичные форматы, но выигрыш от них здесь не определяющий: сообщения о прогрессе содержат немного числовых полей, и затраты на их сериализацию пренебрежимо малы по сравнению со временем самих вычислений [22]. Для оценки объёма: финальная популяция размером 50×10 содержит 500 чисел, что в JSON составляет около 5–7 КБ — приемлемо для однократной передачи по завершении задачи. В потоке прогресса массив популяции не передаётся: клиент получает лишь номер поколения и лучшее значение.

Клиентская часть написана на языке Dart с использованием фреймворка Flutter. Dart обладает строгой статической типизацией и встроенной поддержкой асинхронного программирования [28]. Особенно важна абстракция потока асинхронных событий: поток сообщений, приходящих по WebSocket, естественно представляется такой последовательностью, на которую приложение подписывается и реагирует по мере поступления данных. Flutter — кроссплатформенный фреймворк, который компилирует приложение в нативный код и использует собственный движок рендеринга [29]. Интерфейс описывается деревом виджетов, и при получении новых данных от сервера перестраивается только изменившаяся его часть, что позволяет плавно обновлять график сходимости в реальном времени. Для рисования контурной карты и трёхмерной поверхности используется низкоуровневый интерфейс рисования на холсте, дающий доступ к отдельным графическим примитивам.

3.2 Реализация вычислительного ядра сервера

Вычислительное ядро сервера отвечает за запуск алгоритмов оптимизации. Оно состоит из трёх частей: реестра методов, набора целевых функций и связующего модуля запуска. Такое разделение позволяет добавлять новые алгоритмы и функции, не меняя остальной код сервера.

Реестр методов — это единая точка, в которой перечислены все доступные алгоритмы. Каждому методу сопоставляется пара из класса алгоритма и класса его конфигурации, а ключом служит строковое имя метода, которое приходит от клиента в запросе на запуск. Содержимое реестра показано в листинге 1.

Листинг 1: Реестр методов оптимизации

```
1 METHODS = {
2     "bbo.classic": (ClassicBBO, ClassicBBOConfig),
3     "bbo.classic.modified": (ClassicBBO_Modified, BBOConfigModified),
4     "bbo.sinusoidal": (BBO_Sinusoidal, SinBBOConfig),
5     "bbo.sinusoidal.modified": (BBO_Sinusoidal_Modified,
6         BBOConfigSinModified),
7     "bbo.mixed": (BBO_Mixed, MixedBBOConfig),
8     "bbo.mixed.modified": (BBO_Mixed_Modified, BBOConfigMixedModified),
9     "pso": (PSO, PSOConfig),
10 }
```

Чтобы создать алгоритм, фабрика находит в реестре пару по имени метода, собирает объект конфигурации из присланных параметров, дополняя его размерностью и границами области, и возвращает готовый к запуску экземпляр алгоритма. Если имя метода в реестре отсутствует, фабрика сообщает об ошибке. Благодаря этому добавление нового алгоритма сводится к его регистрации в реестре — остальной код изменять не требуется.

Целевая функция создаётся отдельно. Система поддерживает два вида функций. Встроенные функции — сфера, Растригина, Экли и Букина N.6 — выбираются по имени из заранее заданного набора. Кроме того, пользователь может задать свою функцию в виде математического выражения от координат. Выполнять такое выражение напрямую небезопасно: в нём может оказаться обращение к файловой системе или иной нежелательный код. Поэтому выражение не вычисляется как обычный код, а сначала разбирается в синтаксическое дерево, которое проверяется на допустимость. В выражении разрешены только арифметические операции и функции из ограничен-

ного набора — квадратный корень, экспонента, логарифм, тригонометрические функции и подобные. Любой вызов постороннего имени или обращение к атрибуту объекта приводят к отклонению выражения. Только прошедшее проверку выражение компилируется в функцию. При вычислении ей доступны лишь координаты точки и функции из разрешённого набора, а доступ к встроенным средствам языка закрыт, поэтому даже если какая-то опасная конструкция минует проверку, выполнить её будет нечем [24].

Связующий модуль запуска объединяет эти части и выполняет один запуск от начала до конца. Он разбирает пришедшие параметры задачи, строит границы области поиска, создаёт целевую функцию и алгоритм, после чего запускает оптимизацию. Во время работы алгоритм вызывает переданную ему функцию обратного вызова после каждого поколения; через неё модуль запуска передаёт данные о прогрессе наружу и одновременно проверяет, не поступила ли команда отмены. Если отмена запрошена, выполнение прерывается. По завершении модуль приводит результат к виду, пригодному для передачи по сети: массивы значений преобразуются в списки, добавляются сведения о затраченном времени и использованном методе.

3.3 Реализация очереди задач и обработчика

Поскольку сервер может обслуживать несколько клиентов сразу, а каждый запуск оптимизации занимает заметное время, задачи не выполняются немедленно по поступлении, а проходят через очередь. За постановку задач в очередь и их извлечение отвечает очередь задач, а за само выполнение — обработчик.

Очередь реализована на основе приоритетной очереди из библиотеки `asyncio`. Каждый элемент очереди — это тройка из числового приоритета, порядкового номера постановки и идентификатора задачи. Поддерживаются три уровня приоритета: высокий, обычный и низкий; при равных приоритетах задачи выполняются в порядке поступления, для чего и служит порядковый номер. Сам объект задачи хранится в отдельном реестре, а в очереди лежит только его идентификатор, по которому обработчик получает задачу целиком.

При постановке задачи очередь решает ещё две вспомогательные проблемы. Первая — защита от повторной постановки. Клиент присваивает каждому запросу уникальный идентификатор; если из-за нестабильного со-

единения запрос придёт повторно, очередь обнаружит совпадение по этому идентификатору и вернёт уже существующую задачу, не создавая новую и не запуская повторных вычислений. Вторая — защита от переполнения: число одновременно ожидающих задач ограничено, и при достижении предела новая постановка отклоняется с сообщением об ошибке. Кроме того, по идентификатору задачи можно узнать её позицию в очереди и приблизительное время ожидания, а также отменить задачу — как ожидающую, так и уже выполняемую.

Выполнением задач занимается обработчик. Таких обработчиков несколько: при запуске сервера создаётся пул из нескольких параллельных обработчиков, и их число задаётся в настройках сервера. Каждый из них работает в постоянном цикле: извлекает из очереди очередную задачу, переводит её в состояние выполнения, уведомляет клиента о начале и запускает оптимизацию. Поскольку несколько обработчиков берут задачи из общей очереди, сервер может выполнять сразу несколько задач одновременно. Сам алгоритм оптимизации вычислительно нагружен и выполняется без точек ожидания, поэтому его нельзя запускать прямо в асинхронном цикле событий — это заблокировало бы обработку остальных соединений. Поэтому алгоритм запускается в отдельном потоке из пула, а асинхронный цикл тем временем продолжает обслуживать других клиентов [24].

На время выполнения задаётся ограничение по времени. Запуск алгоритма обёрнут в ожидание с таймаутом: если вычисление не завершилось за отведённый срок, оно прерывается, а задача переходит в состояние превышения времени. Аналогично обрабатываются и другие исходы — успешное завершение, ошибка во время вычислений и отмена пользователем; каждому из них соответствует свой переход в терминальное состояние и своё сообщение клиенту.

Отдельного внимания заслуживает передача прогресса. Алгоритм выполняется в рабочем потоке, а отправка сообщений клиенту идёт в асинхронном цикле, поэтому напрямую передавать данные между ними нельзя. Прогресс переправляется через потокобезопасную очередь: после каждого поколения рабочий поток помещает в неё данные, а отдельная задача в цикле событий извлекает их и отправляет клиенту. Такая передача поддерживает несколько режимов: сообщать о каждом поколении, о каждом k -м или

не сообщать вовсе, что позволяет снизить плотность потока сообщений для длительных запусков. Когда вычисление завершается, в очередь прогресса помещается признак конца, по которому отправка прогресса корректно останавливается.

Связь очереди с клиентами обеспечивает обработчик соединений. Для каждого подключившегося клиента создаётся отдельный обработчик, который ведёт диалог с ним по протоколу обмена. Сначала он принимает приветственное сообщение и в ответ сообщает клиенту о возможностях сервера: какие методы оптимизации доступны, какие виды целевых функций поддерживаются, какие есть режимы передачи прогресса и каков предельный размер сообщения. Если первым приходит не приветствие, соединение отклоняется. После рукопожатия обработчик входит в цикл чтения сообщений и направляет каждое из них в зависимости от типа: запрос на запуск ставит задачу в очередь, запрос на отмену снимает задачу, запрос состояния возвращает сведения о задаче, а служебное сообщение проверки связи подтверждается ответом. Сообщения неизвестного типа или с некорректными данными не прерывают работу — в ответ отправляется сообщение об ошибке, и обработчик продолжает принимать следующие. Когда клиент отключается, обработчик отменяет все его незавершённые задачи, чтобы сервер не тратил ресурсы на вычисления, результат которых уже некому получить.

Перед постановкой в очередь параметры задачи проверяются на корректность. Проверка охватывает структуру сообщения и допустимость значений: имя метода должно быть известным, размерность и размер популяции — положительными, границы области — согласованными. Некорректный запрос отклоняется с понятным сообщением об ошибке ещё до запуска вычислений, что защищает сервер от заведомо неверных задач.

3.4 Реализация алгоритмов оптимизации

Все семь методов реализованы по общей схеме. Каждый алгоритм оформлен как отдельный класс, который получает при создании три вещи: конфигурацию, целевую функцию и функцию обратного вызова для передачи прогресса. Основную работу алгоритм выполняет в методе `optimize`: он инициализирует популяцию, проводит заданное число поколений и возвращает результат — лучшее решение, его значение, историю сходимости и финаль-

ное состояние популяции. Различаются методы тем, что происходит внутри одного поколения.

Параметры каждого метода описаны отдельным классом конфигурации. В нём заданы значения по умолчанию и проверки корректности: например, размер популяции не меньше двух, вероятность мутации лежит в отрезке от нуля до единицы, а число элитных решений меньше размера популяции. Если границы области поиска не заданы явно, они заполняются значениями по умолчанию. Эти проверки выполняются при создании конфигурации, поэтому некорректные параметры отсекаются до запуска вычислений. Случайность всех алгоритмов исходит из одного генератора, создаваемого с заданным начальным значением, — это делает запуски воспроизводимыми, что важно при сравнении методов.

Рассмотрим реализацию классического ВВО. Одно его поколение показано в листинге 2. Сначала популяция сортируется по значению целевой функции, после чего по рангам вычисляются скорости эмиграции и иммиграции. Затем для каждого неэлитного решения выполняется покомпонентная иммиграция: компонента заменяется значением донора с вероятностью, равной скорости иммиграции, а донор выбирается случайно с вероятностью, пропорциональной скорости эмиграции. После миграции применяется мутация, а в конце поколения лучшие решения возвращаются в популяцию вместо худших потомков.

Листинг 2: Одно поколение классического ВВО

```
1 order = np.argsort(fitness)
2 X, fitness = X[order], fitness[order]
3 best_f, best_x = float(fitness[0]), X[0].copy()
4
5 lambdas, mus = self._rank_based_migration_rates(self.cfg.pop_size)
6 donor_probs = lambdas[::-1] / lambdas[::-1].sum()
7 donor_cdf = np.cumsum(donor_probs)
8
9 Z = X.copy()
10 for k in range(self.cfg.elite_count, self.cfg.pop_size):
11     imm_flags = self.rng.random(self.cfg.dims) < mus[::-1][k]
12     for s in np.where(imm_flags)[0]:
13         j = self._roulette_select(donor_cdf)
14         Z[k, s] = X[j, s]
15     mut_flags = self.rng.random(self.cfg.dims) < self.cfg.mutation_rate
16     Z[k, mut_flags] = self.low[mut_flags] + self.rng.random(mut_flags.
        sum()) * self.span[mut_flags]
```

```

17
18 fit_Z = self._evaluate_population(self._clip(Z))
19 worst = np.argsort(fit_Z)[-self.cfg.elite_count:]
20 Z[worst], fit_Z[worst] = X[:self.cfg.elite_count], fitness[:self.cfg.
    elite_count]
21 X, fitness = Z, fit_Z

```

Варианты с синусоидальной и смешанной моделями миграции построены по тому же циклу. Отличие у них только в одном месте — в способе вычисления скоростей λ_r и μ_r : каждый вариант рассчитывает их по своей формуле, приведённой в подразделе 1.3. Всё остальное — сортировка, покомпонентная иммиграция, мутация, элитарность — совпадает с классическим вариантом.

Модифицированные версии устроены сложнее. К базовому циклу в них добавлены три механизма адаптации, описанные в подразделе 1.4, и поколение складывается из большего числа шагов. Его структура показана в листинге 3. В начале поколения вычисляется текущая вероятность мутации, которая линейно убывает от начального значения к конечному. Затем проверяется счётчик стагнации: если лучшее решение не улучшалось заданное число поколений, выполняется частичный рестарт, а вероятность мутации временно повышается. После этого идут миграция и мутация, применяется элитарность, и при необходимости запускается локальный поиск. В конце поколения счётчик стагнации обновляется в зависимости от того, удалось ли улучшить лучшее решение.

Листинг 3: Одно поколение модифицированного ВВО

```

1 mutation_rate = self._current_mutation_rate(gen)
2 if stagnation >= self.cfg.stagnation_generations:
3     X, fit = self._restart_partially(X, fit)
4     mutation_rate = min(self.cfg.mutation_start,
5                          mutation_rate * self.cfg.mutation_boost_factor)
6     stagnation = 0
7
8 X_new = self._migration_step(X, fit)
9 X_new = self._mutation_step(X_new, mutation_rate)
10 fit_new = self._evaluate(X_new)
11 X, fit = self._apply_elitism(X, fit, X_new, fit_new)
12
13 if self.cfg.enable_local_search and (gen + 1) % self.cfg.
14     local_search_interval == 0:
15     X, fit = self._local_search(X, fit, gen)
16
17 cur_best = float(fit[np.argmin(fit)])

```

```

17 if cur_best < best_f:
18     best_f, stagnation = cur_best, 0
19 else:
20     stagnation += 1

```

Частичный рестарт реализован как замена доли худших решений новыми случайными. Число заменяемых решений определяется заданной долей от размера популяции, при этом элитные решения защищены от замены. Локальный поиск уточняет несколько лучших решений: к каждому из них последовательно добавляется случайное гауссовское возмущение, масштаб которого пропорционален ширине области поиска и убывает с номером поколения. Если возмущённая точка оказывается лучше, она принимается, иначе отвергается. Так локальный поиск повышает точность вблизи найденных минимумов, не затрагивая остальную популяцию.

Метод роя частиц реализован матрично: вся популяция обновляется за одну группу операций над массивами, без цикла по частицам. Одна итерация PSO показана в листинге 4. Скорости всех частиц пересчитываются сразу: к инерционной составляющей добавляются когнитивная, направленная к личному лучшему положению, и социальная, направленная к глобально лучшему. Затем скорости ограничиваются по модулю, положения сдвигаются и отсекаются по границам области, после чего обновляются личные и глобально лучшее решения.

Листинг 4: Одна итерация метода роя частиц

```

1 w = self._inertia_weight(t)
2 r1, r2 = self.rng.random((N, n)), self.rng.random((N, n))
3
4 cognitive = self.cfg.c1 * r1 * (pbest_pos - X)
5 social    = self.cfg.c2 * r2 * (gbest_pos - X)
6 V = w * V + cognitive + social
7 V = np.clip(V, -self.vmax, self.vmax)
8
9 X = np.clip(X + V, self.low, self.high)
10 fit = self._evaluate(X)
11
12 improved = fit < pbest_fit
13 pbest_pos[improved], pbest_fit[improved] = X[improved], fit[improved]
14 g_idx = int(np.argmin(pbest_fit))
15 if pbest_fit[g_idx] < gbest_fit:
16     gbest_fit, gbest_pos = float(pbest_fit[g_idx]), pbest_pos[g_idx].
        copy()

```

Использование операций над массивами NumPy здесь принципиально. Вычисление целевой функции сразу для всей популяции, сортировка, отбор и пересчёт скоростей выполняются как единые операции над матрицами, а не как циклы на чистом Python [24]. Для метода роя частиц это позволяет полностью избавиться от цикла по частицам; в ВВО покомпонентная иммиграция всё же требует перебора, поскольку для каждой компоненты донор выбирается независимо, но оценка популяции и мутация остаются векторными. При вычислении целевой функции учитывается её вид: сначала функция вызывается сразу для всей матрицы решений, и лишь если так вычислить не удалось — например, для пользовательского выражения, рассчитанного поточечно, — происходит переход к поэлементному вычислению. После каждого поколения все методы вызывают функцию обратного вызова, передавая ей номер поколения и лучшее найденное значение, — именно эти данные затем уходят клиенту в потоке прогресса.

3.5 Реализация мобильного клиента

Мобильный клиент построен по слоистой схеме. Нижний слой отвечает за связь с сервером по протоколу обмена, средний хранит состояние экранов и обрабатывает события, верхний отображает интерфейс. Дополнительно выделены слой моделей данных, описывающий структуру сообщений, и слой хранения, отвечающий за локальную базу истории. Такое разделение позволяет менять одну часть, не затрагивая остальные.

3.5.1 Клиент протокола

Связь с сервером инкапсулирована в отдельном классе, который скрывает детали соединения WebSocket и предоставляет приложению единый поток событий. Сообщения от сервера разбираются и превращаются в типизированные события, публикуемые в широковещательный поток; на него подписываются те части приложения, которым нужны данные с сервера. Приём и маршрутизация сообщений показаны в листинге 5: строка разбирается в конверт, из него извлекаются тип и полезная нагрузка, после чего по типу создаётся соответствующее событие.

Листинг 5: Приём и маршрутизация сообщений на клиенте

```
1 void _onMessage(String raw) {  
2     final envelope = parseEnvelope(raw);
```

```

3   final type = getType(envelope);
4   final payload = getPayload(envelope);
5
6   switch (type) {
7       case MessageTypes.jobAccepted:
8           _eventController.add(WsJobAccepted(JobAccepted.fromPayload(
9               payload)));
10          case MessageTypes.jobStarted:
11              _eventController.add(WsJobStarted(JobStarted.fromPayload(payload)
12                  ));
13          case MessageTypes.jobProgress:
14              _eventController.add(WsJobProgress(JobProgress.fromPayload(
15                  payload)));
16          case MessageTypes.jobResult:
17              _eventController.add(WsJobResult(JobResult.fromPayload(payload)))
18                  ;
19          case MessageTypes.jobFinished:
20              _eventController.add(WsJobFinished(JobFinished.fromPayload(
21                  payload)));
22          case MessageTypes.error:
23              _eventController.add(WsError(ProtocolError.fromPayload(payload)))
24                  ;
25      }
26  }

```

Клиент протокола устойчив к обрывам связи. Если соединение неожиданно закрывается, он пытается восстановить его автоматически, причём задержка между попытками растёт по степенному закону — от одной секунды до тридцати, — чтобы не нагружать сеть частыми попытками. Пока соединение установлено, клиент периодически посылает служебное сообщение проверки связи. При подключении он отправляет серверу приветствие, в котором сообщает о желании получать поток прогресса. Намеренное отключение пользователем отличается от обрыва специальным признаком: при нём автоматическое восстановление не запускается. Кроме приёма событий клиент предоставляет методы для отправки задачи, запроса её состояния и отмены.

3.5.2 Модели данных и конверт сообщения

Сообщения протокола разбираются и собираются в отдельном слое моделей. Каждому типу сообщения соответствует свой класс с методом разбора из полезной нагрузки, благодаря чему остальной код работает с типизированными объектами, а не с необработанными словарями. Формирование исходя-

щих сообщений вынесено в общую функцию построения конверта, которая добавляет служебные поля заголовка — версию протокола, тип, уникальный идентификатор сообщения, метку времени и поля для связи сообщений и идентификации задачи. Эта функция показана в листинге 6.

Листинг 6: Построение конверта исходящего сообщения

```
1 Map<String, dynamic> buildEnvelope({
2   required String type,
3   required Map<String, dynamic> payload,
4   String? clientReqId,
5   String? jobId,
6 }) {
7   return {
8     'header': {
9       'v': 1,
10      'type': type,
11      'msg_id': _uuid.v4(),
12      'ts': DateTime.now().toUtc().toIso8601String(),
13      'trace': {'client_req_id': clientReqId, 'job_id': jobId},
14    },
15    'payload': payload,
16  };
17 }
```

3.5.3 Сборка конфигурации задачи

Параметры эксперимента, задаваемые на экране настройки, собираются из нескольких независимых частей: описания целевой функции, постановки задачи, выбранного метода, режима потоковой передачи и состава выходных данных. Описание целевой функции хранит её вид — встроенная или заданная выражением — и соответствующие поля. Постановка задачи задаёт размерность и границы области поиска, которые могут быть едиными для всех координат или различаться по каждой. Параметры метода зависят от выбранного алгоритма: у каждого метода свой набор полей, например размер популяции и вероятность мутации для ВВО или коэффициенты для метода роя частиц. Каждая часть умеет преобразовывать себя в представление для передачи, а перед отправкой все части объединяются в единую полезную нагрузку запроса на запуск. Так на клиенте формируется тот самый запрос, который сервер затем разбирает и исполняет.

3.5.4 Управление состоянием

Средний слой реализован по схеме однонаправленного потока данных: каждому экрану соответствует объект управления состоянием, который подписывается на поток событий клиента протокола, обрабатывает их и порождает новое неизменяемое состояние, на которое реагирует интерфейс. Таких объектов несколько: один отвечает за подключение к серверу, другой — за сборку конфигурации, третий — за наблюдение за ходом задачи, четвёртый — за работу с историей. Наиболее показателен объект состояния экрана прогресса. При создании он отправляет серверу задачу и подписывается на события, а получая сообщения о ходе вычисления, инкрементально накапливает историю лучших значений, как показано в листинге 7. Новые значения дописываются в конец уже накопленной истории, что и позволяет графику сходимости достраиваться в реальном времени, не перестраиваясь заново.

Листинг 7: Накопление истории прогресса

```
1 case WsJobProgress(:final data):
2   final newHistory = List<double>.from(state.historyBestF);
3   if (data.progress.historyBestFTail != null) {
4     for (final v in data.progress.historyBestFTail!) {
5       if (newHistory.isEmpty || newHistory.last != v) newHistory.add(v)
6       ;
7     }
8   } else {
9     newHistory.add(data.progress.bestF);
10  }
11  emit(state.copyWith(
12    phase: JobPhase.running,
13    iteration: data.progress.iteration,
14    bestF: data.progress.bestF,
15    historyBestF: newHistory,
16  ));
```

Этот же объект отслеживает фазу задачи: ожидание в очереди, начало выполнения, ход вычисления и завершение. Когда от сервера приходит сообщение о завершении, фаза переключается в одно из конечных состояний — успех, ошибка, отмена или превышение времени, — и интерфейс соответственно меняет вид. Отдельно накапливается история лучших решений в пространстве, которая затем используется для построения траектории на трёхмерной визуализации.

3.5.5 Локальное хранение истории

История завершённых экспериментов хранится на устройстве в локальной базе данных. Результат каждого запуска сериализуется в текстовое представление и сохраняется отдельной записью вместе с метаданными — методом, целевой функцией, размерностью, итоговым значением и временем проведения. При открытии истории записи считываются обратно и восстанавливаются в типизированные объекты, а метаданные позволяют отбирать и фильтровать запуски. Хранение на самом устройстве делает историю доступной независимо от того, подключён ли клиент к серверу.

3.6 Пользовательский интерфейс приложения

В этом подразделе показано, как выглядит готовое приложение. Интерфейс выдержан в едином стиле: общая цветовая гамма, крупные элементы управления, удобные для работы на сенсорном экране, и привычные для мобильных приложений компоненты — выпадающие списки, переключатели и ползунки. Экраны рассматриваются в том порядке, в каком пользователь обычно проходит через них при работе.

3.6.1 Подключение к серверу

С этого экрана начинается работа. Пользователь вводит адрес и порт сервера и нажимает кнопку подключения; под кнопкой отображается текущее состояние соединения. Адрес последнего использованного сервера сохраняется, поэтому при повторном запуске его не нужно вводить заново. Экран подключения показан на рисунке 10.

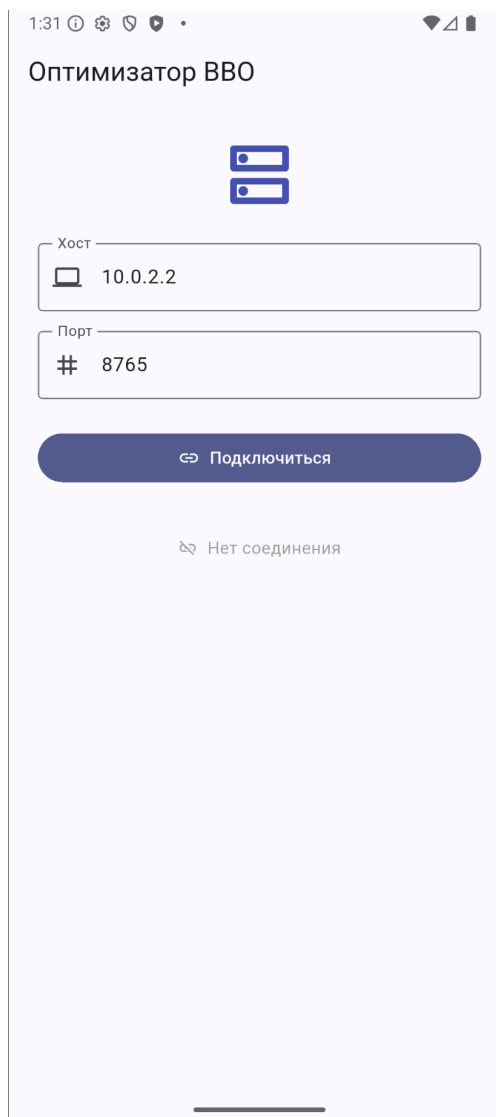


Рисунок 10 – Экран подключения к серверу

3.6.2 Настройка эксперимента

После подключения открывается основной экран настройки. Он разбит на смысловые блоки: выбор целевой функции, постановка задачи, метод оптимизации, режим потоковой передачи и состав выходных данных. В блоке целевой функции выбирается встроенная функция или вводится своё выражение, а переключатель задаёт минимизацию или максимизацию. В блоке постановки задачи ползунком задаётся размерность, а в полях ниже — границы области поиска. В блоке метода выбирается алгоритм, и для него появляется свой набор параметров: размер популяции, число поколений, вероятность мутации и другие. Поскольку настроек много, экран прокручивается; внизу находится кнопка запуска оптимизации. Вид экрана показан на рисунке 11.

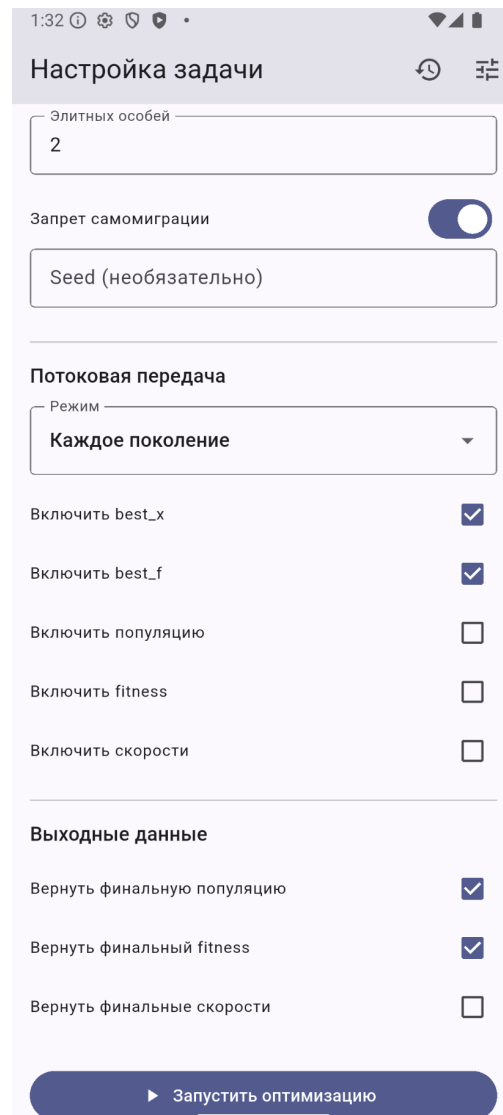
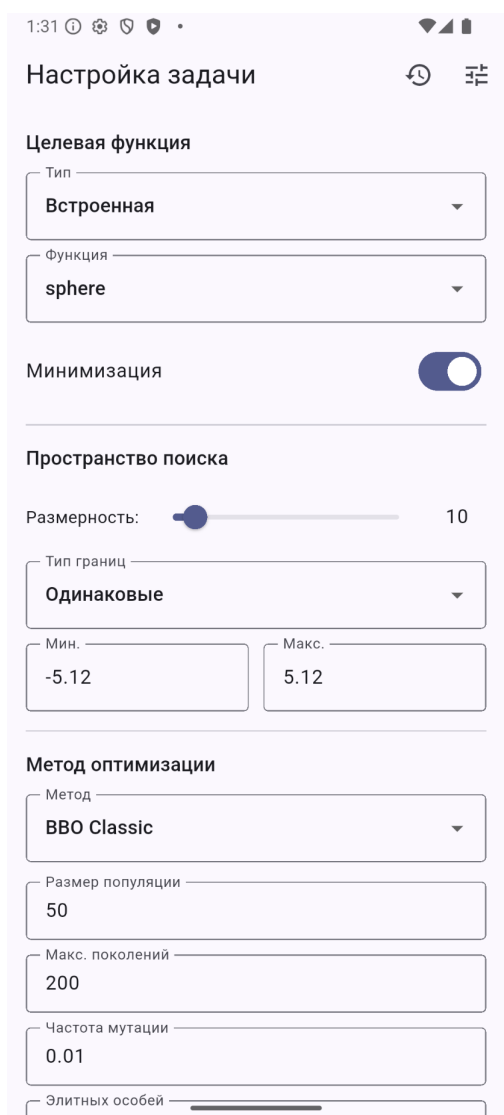


Рисунок 11 – Экран настройки эксперимента: выбор функции и задачи (слева), параметры метода и потоковой передачи (справа)

3.6.3 Выполнение и наблюдение за прогрессом

После запуска приложение переходит на экран выполнения. Вверху находится индикатор прогресса с номером текущего поколения и долей выполнения, ниже — лучшее найденное значение и затраченное время. Основную часть экрана занимает график сходимости, который достраивается в реальном времени по мере поступления данных с сервера. Внизу расположена кнопка отмены, позволяющая прервать вычисление, не дожидаясь его окончания. Экран выполнения показан на рисунке 12.

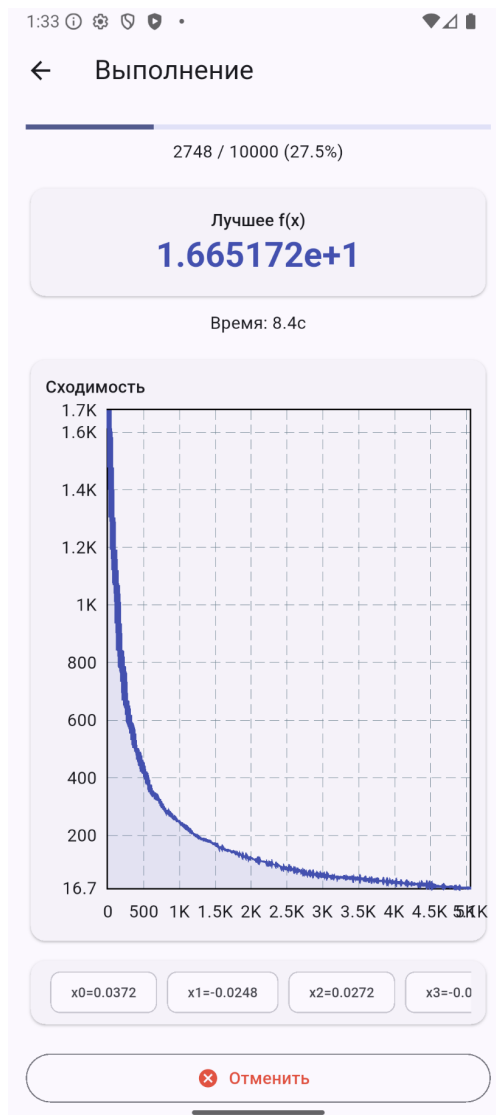


Рисунок 12 – Экран выполнения с графиком сходимости в реальном времени

3.6.4 Просмотр результата

По завершении вычисления открывается экран результата. Вверху выводится лучшее найденное значение целевой функции, ниже — координаты лучшего решения и сводка показателей: метод, число поколений и вычислений функции, затраченное время. Далее расположен график сходимости за весь запуск, а под ним — визуализация пространства поиска в виде контурной карты, на которой отмечены лучшее решение и ближайшие к нему особи. Здесь же находятся кнопки построения анимации и перехода к трёхмерной визуализации. Вид экрана показан на рисунке 13.

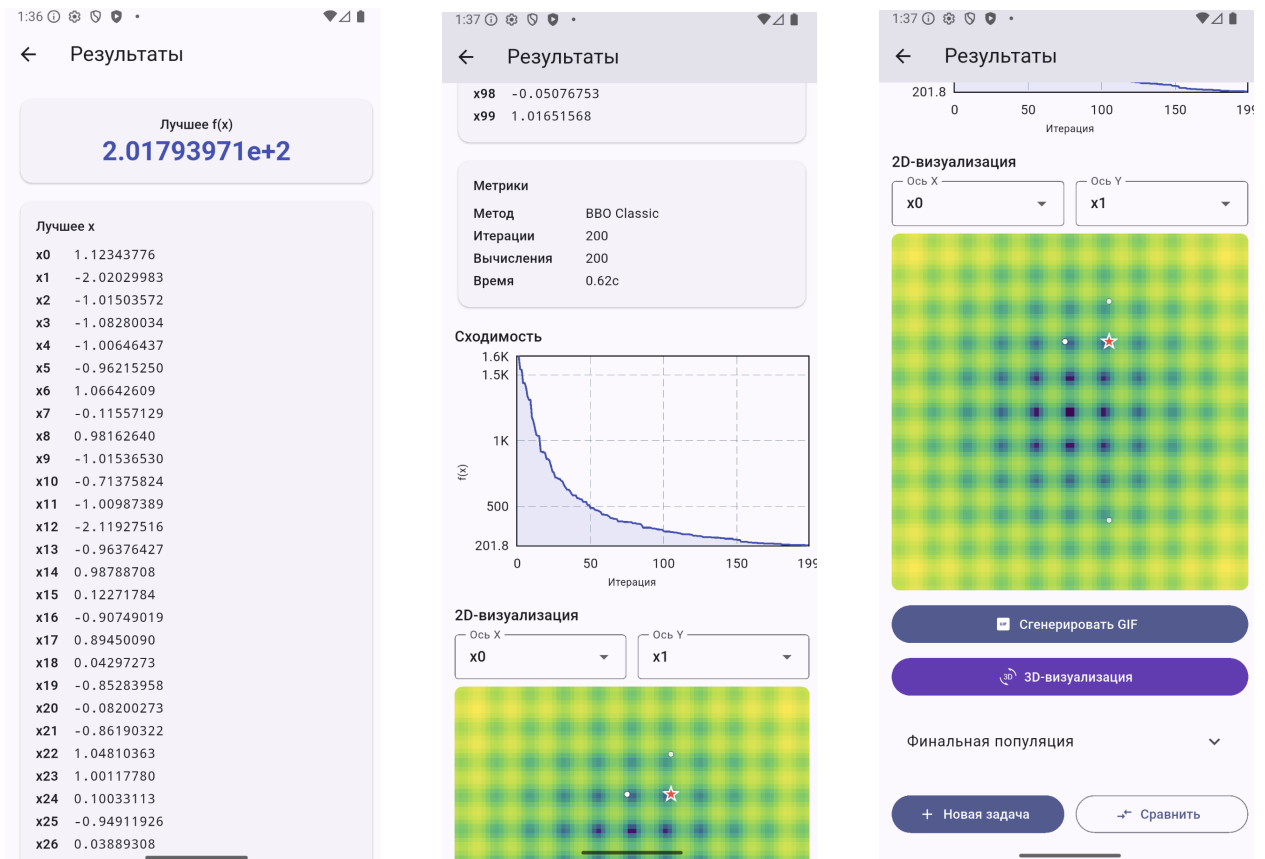


Рисунок 13 – Экран результата: лучшее решение (слева), показатели и график сходимости (в центре), контурная карта пространства поиска (справа)

По нажатию кнопки строится анимация: контурная карта, на которой популяция смещается от поколения к поколению. Готовая анимация показывается прямо на экране, как на рисунке 14.

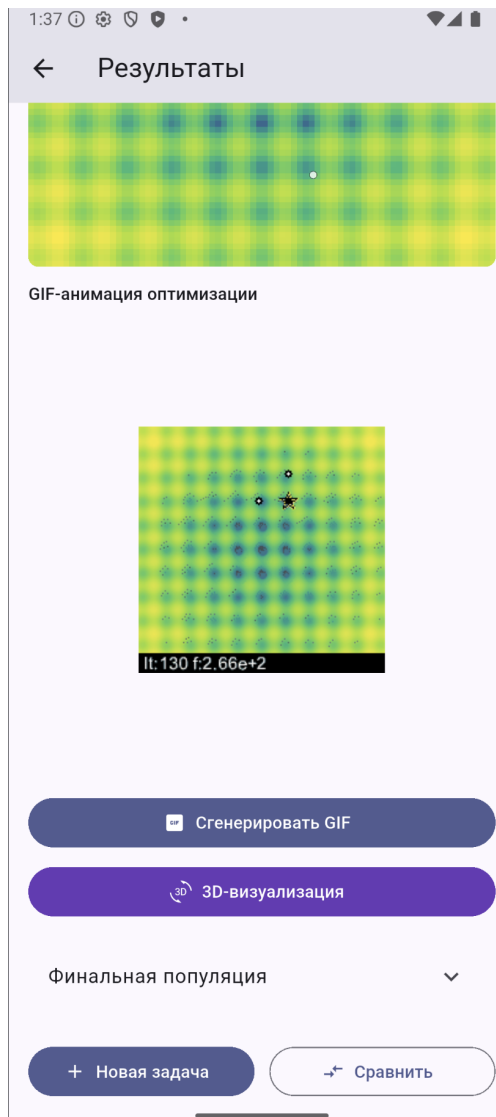


Рисунок 14 – Анимация процесса оптимизации на экране результата

3.6.5 Трёхмерная визуализация

Отдельный экран показывает целевую функцию трёхмерной поверхностью. Высота точки соответствует значению функции, на поверхности отмечены лучшее решение и путь, по которому оно улучшалось. Поверхность можно вращать движением пальца, рассматривая рельеф под разными углами, а выпадающие списки сверху позволяют выбрать, какие две координаты отложить по осям. Экран трёхмерной визуализации показан на рисунке 15.

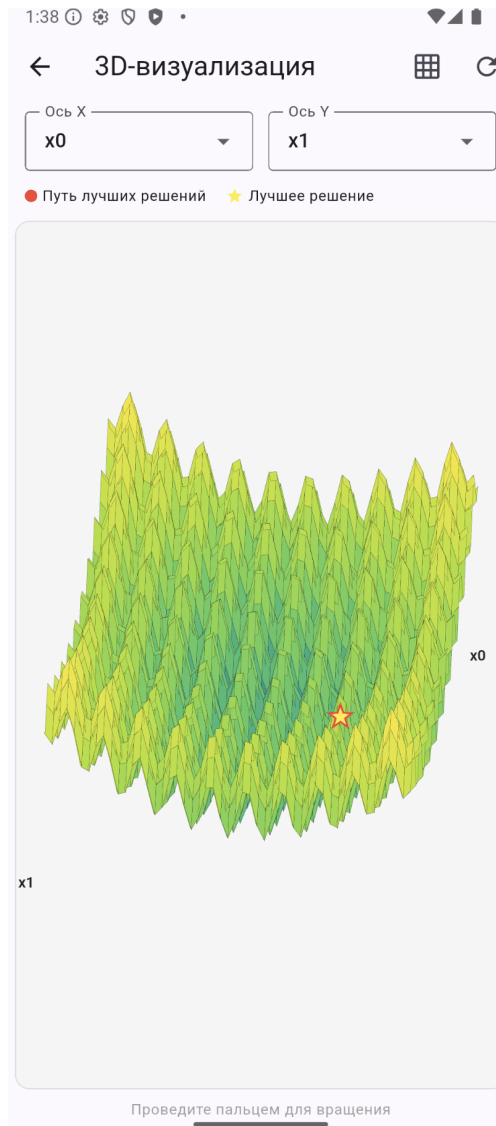


Рисунок 15 – Экран трёхмерной визуализации целевой функции

3.6.6 История и сравнение запусков

Все завершённые эксперименты попадают в историю. На её экране они показаны списком, где для каждого запуска указаны метод, целевая функция, размерность, итоговое значение и время проведения; список можно фильтровать по методу и функции. Отметив несколько запусков, пользователь переходит к их сравнению. На экране сравнения выводятся совмещённый график сходимости выбранных запусков, таблица их показателей и столбчатая диаграмма лучших значений. Оба экрана показаны на рисунке 16.

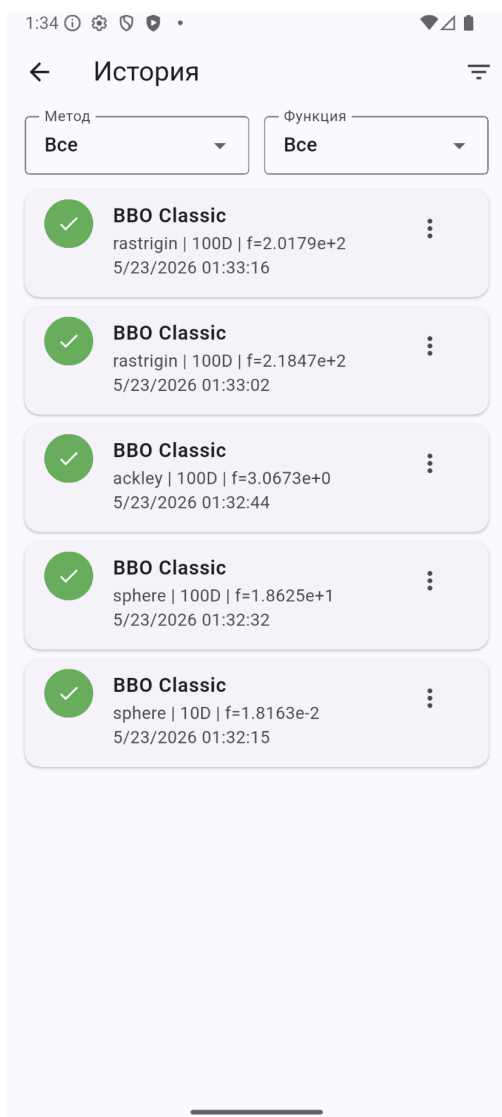


Рисунок 16 – Экран истории экспериментов (слева) и экран сравнения запусков (справа)

3.7 Реализация визуализации результатов

Способы визуализации, описанные в подразделе 1.6, реализованы на стороне клиента. График сходимости строится готовым средством построения графиков, а контурная карта, трёхмерная поверхность и кадры анимации рисуются вручную через низкоуровневый интерфейс рисования на холсте, который даёт доступ к отдельным графическим примитивам. Ниже рассмотрена реализация каждого вида.

3.7.1 График сходимости

График сходимости показывает, как убывает лучшее найденное значение от поколения к поколению. Он строится готовым средством построения графиков, которому передаётся список накопленных значений. На экране вы-

полнения этот список пополняется по мере прихода сообщений о прогрессе, поэтому график достраивается в реальном времени. Когда значения функции убывают на несколько порядков, по оси ординат применяется логарифмическая шкала, иначе нижняя часть кривой сжимается у нуля и становится нечитаемой. На экране результата и на экране сравнения тот же график строится по полной истории запуска, причём при сравнении на одни оси накладывается несколько кривых, различающихся цветом.

3.7.2 Контурная карта

Контурная карта строится вручную. Область поиска разбивается на равномерную сетку, для каждой ячейки вычисляется значение целевой функции, которое переводится в цвет по выбранной цветовой шкале, и ячейка закрашивается прямоугольником. Значение переводится в цвет через логарифмическую нормировку, что делает различимыми и глубокие минимумы, и пологие участки. Поверх карты наносятся положения особей популяции и отдельно отмечается лучшее решение. Основная часть рисования показана в листинге 8.

Листинг 8: Построение контурной карты

```
1 for (int j = 0; j < resolution; j++) {
2   for (int i = 0; i < resolution; i++) {
3     final val = grid.values[j][i];
4     final t = range > 0
5       ? (math.log(val - minVal + 1) / math.log(range + 1)).clamp(0.0,
6         1.0)
7       : 0.0;
8     paint.color = ColorMaps.viridis(t);
9     canvas.drawRect(
10      Rect.fromLTWH(i * cellW, j * cellH, cellW + 0.5, cellH + 0.5),
11      paint,
12    );
13  }
```

Поскольку построение карты требует вычислить функцию во всех узлах сетки, оно выполняется один раз при открытии экрана, а при перерисовке используются уже полученные значения. При размерности задачи больше двух по осям откладываются две выбранные пользователем координаты, а остальные фиксируются, что даёт сечение функции в выбранной плоскости.

3.7.3 Трёхмерная поверхность

Трёхмерная поверхность строится по той же сетке значений функции. Узлы сетки образуют четырёхугольные грани, которые проектируются на плоскость экрана. Проекция выполняется по формулам поворота, приведённым в подразделе 1.6: координаты каждого узла нормируются к единому диапазону и последовательно поворачиваются вокруг вертикальной и горизонтальной осей в соответствии с формулами (13) и (14), после чего две полученные координаты дают положение точки на экране. Реализация проекции показана в листинге 9.

Листинг 9: Проекция точки поверхности на экран

```
1 Offset project(double px, double py, double pz) {  
2     final nx = 2 * (px - xMin) / (xMax - xMin) - 1;  
3     final ny = 2 * (py - yMin) / (yMax - yMin) - 1;  
4     final nz = 2 * (pz - fMin) / fRange - 1;  
5  
6     final rx = nx * cosZ - ny * sinZ;           // rotate around Z  
7     final ry = nx * sinZ + ny * cosZ;  
8     final ry2 = ry * cosX - nz * sinX;         // rotate around X  
9  
10    return Offset(cx + rx * scale, cy - ry2 * scale);  
11 }
```

Чтобы ближние грани правильно перекрывали дальние, применяется алгоритм художника: для каждой грани вычисляется её глубина, грани сортируются от дальних к ближним и рисуются в этом порядке. Цвет грани определяется средним значением функции в её узлах по той же шкале, что и контурная карта. Поверх поверхности наносится путь лучшего решения — накопленная за время поиска последовательность точек, соединённых линией; чтобы не рисовать тысячи точек, путь прореживается. Углы поворота меняются жестом пользователя по экрану, и при каждом изменении поверхность проектируется заново, благодаря чему её можно рассматривать под любым углом.

3.7.4 Анимация процесса оптимизации

Анимация показывает, как популяция смещалась от поколения к поколению. Она собирается кадром в формат GIF. Сначала строится фоновый кадр с контурной картой целевой функции, затем для каждого кадра поверхность наносится положения особей и лучшее решение, а также служебная

строка с номером поколения и текущим лучшим значением. Число кадров фиксировано, поэтому из всей истории берётся равномерная выборка поколений. Готовые кадры собираются в один файл с заданной частотой смены, причём последний кадр показывается дольше остальных, чтобы итоговое расположение популяции было видно. Сборка кадров показана в листинге 10.

Листинг 10: Покадровая сборка анимации

```
1 final step = math.max(1, totalIterations ~/ frameCount);
2 for (int f = 0; f < frameCount; f++) {
3     final iterIndex = math.min(f * step, totalIterations - 1);
4     final progress = (f + 1) / frameCount;
5     final frame = img.Image.from(bgFrame);
6
7     final visibleCount =
8         (population.length * progress).round().clamp(1, population.length
9         );
10    for (int p = 0; p < visibleCount; p++) {
11        final px = _mapCoord(population[p][dimX], xMin, xMax, size);
12        final py = _mapCoord(population[p][dimY], yMin, yMax, size);
13        _drawDot(frame, px, py, 2, img.ColorRgba8(255, 255, 255, 200));
14    }
15    _drawStar(frame, bx, by, starSize, img.ColorRgba8(255, 0, 0, 255));
16    _drawInfoBar(frame, iterIndex, historyBestF[iterIndex], size);
17    frames.add(frame);
18 }
```

Готовая анимация показывается прямо на экране результата и может быть сохранена как обычный файл изображения. Поскольку формат GIF не требует внешних средств воспроизведения и собирается из готовых кадров, он удобен для этой задачи.

4 Аналитическая часть

4.1 Тестирование программного обеспечения

Для проверки правильности работы комплекса применялись два вида тестирования. Серверная часть, содержащая вычислительное ядро и логику обработки задач, покрыта автоматическими тестами. Клиентская часть, основу которой составляет пользовательский интерфейс, проверялась вручную по сценариям использования. Такое разделение объясняется тем, что логику сервера удобно проверять программно на множестве входных данных, а работу интерфейса — наблюдением за поведением приложения на устройстве.

4.1.1 Автоматическое тестирование серверной части

Серверная часть протестирована с помощью библиотеки `pytest` [30]. Всего написано 73 теста, сгруппированных по проверяемым модулям. Тесты охватывают как отдельные функции, так и взаимодействие модулей между собой. Ниже перечислены группы тестов и то, что они проверяют.

1. Тесты очереди задач. Проверялась постановка задач в очередь с учётом приоритетов, извлечение в правильном порядке, отмена ожидающих и выполняемых задач, защита от повторной постановки по идентификатору запроса и от переполнения очереди. Эта группа содержит 14 тестов.
2. Тесты целевых функций. Проверялась правильность вычисления встроенных функций в известных точках, разбор и вычисление пользовательских выражений, а также отклонение небезопасных и некорректных выражений. Эта группа содержит 15 тестов.
3. Тесты протокола обмена. Проверялись построение и разбор конверта сообщения, сохранение служебных полей заголовка и корректная обработка всех типов сообщений. Эта группа содержит 11 тестов.
4. Тесты реестра методов. Проверялось, что все семь методов зарегистрированы, что алгоритм создаётся по имени метода с нужными параметрами и что обращение к неизвестному методу приводит к ошибке. Эта группа содержит 4 теста.
5. Тесты модуля запуска. Проверялся полный прогон оптимизации: запуск алгоритма, передача данных о прогрессе, прерывание по команде отме-

ны и приведение результата к виду для передачи по сети. Эта группа содержит 7 тестов.

6. Тесты валидации. Проверялось, что корректные параметры задачи проходят проверку, а некорректные — неизвестный метод, неположительная размерность, несогласованные границы — отклоняются с понятным сообщением. Эта группа содержит 14 тестов.
7. Тесты соединения. Проверялась работа обработчика соединений как по отдельности, так и в связке с очередью: рукопожатие, маршрутизация сообщений, постановка задачи через соединение и получение ответов. Эта группа содержит 8 тестов.

Большинство тестов устроены просто: они вызывают проверяемую функцию и сравнивают результат с ожидаемым. В качестве примера в листинге 11 приведён тест, который проверяет, что в реестре зарегистрированы все семь методов оптимизации.

Листинг 11: Пример теста: проверка состава реестра методов

```
1 def test_all_methods_registered(self):
2     expected = {
3         "bbo.classic", "bbo.classic.modified",
4         "bbo.sinusoidal", "bbo.sinusoidal.modified",
5         "bbo.mixed", "bbo.mixed.modified",
6         "pso",
7     }
8     assert set(METHODS.keys()) == expected
```

Все 73 теста выполняются успешно. Это подтверждает, что вычислительное ядро, очередь, протокол и логика обработки задач работают так, как задумано, и что изменения в коде не нарушают уже проверенное поведение.

4.1.2 Функциональное тестирование клиента

Клиентская часть проверялась вручную: на устройстве последовательно выполнялись основные сценарии использования, и поведение приложения сравнивалось с ожидаемым. Тестирование охватывало все экраны и основные действия пользователя.

1. Сценарий подключения к серверу. Проверялось, что приложение устанавливает соединение по введённым адресу и порту, отображает текущее состояние соединения и восстанавливает его при кратковременном

обрыве. При неверном адресе приложение сообщает об ошибке и не зависает.

2. Сценарий настройки и запуска задачи. Проверялось, что на экране настройки можно выбрать метод и целевую функцию, задать размерность, границы и параметры метода, а также ввести собственное выражение. После запуска приложение переходит к экрану выполнения, а недопустимые значения параметров не позволяют начать вычисление.
3. Сценарий наблюдения за прогрессом. Проверялось, что во время вычисления приложение показывает номер текущего поколения и лучшее значение, а график сходимости достраивается в реальном времени по мере поступления данных с сервера. Кнопка отмены прерывает вычисление, и приложение корректно обрабатывает это состояние.
4. Сценарий просмотра результата. Проверялось, что после завершения задачи отображаются лучшее решение, показатели качества и график сходимости, а также строятся контурная карта, трёхмерная поверхность и анимация. Поверхность вращается жестом, а выбор координат по осям меняет отображаемое сечение.
5. Сценарий работы с историей. Проверялось, что завершённые эксперименты сохраняются в историю, список фильтруется по методу и функции, а любую запись можно открыть и снова увидеть её результат. История сохраняется между запусками приложения.
6. Сценарий сравнения запусков. Проверялось, что несколько выбранных из истории запусков выводятся вместе на совмещённом графике сходимости, в таблице показателей и на столбчатой диаграмме, а сравнение остаётся читаемым при разных методах и функциях.

В ходе ручного тестирования все перечисленные сценарии отработали в соответствии с ожиданиями. Совместно с автоматическими тестами серверной части это позволяет считать, что комплекс работает корректно и готов к проведению вычислительных экспериментов.

4.2 Вычислительные эксперименты и анализ результатов

Главная цель экспериментов — сравнить реализованные методы между собой и оценить, как на их работу влияют модификации базового алгоритма.

Для этого все семь методов запускались на наборе тестовых функций при разных размерностях задачи.

4.2.1 Методика экспериментов

В качестве тестовых использовались четыре функции, описанные в подразделе 1.1: сфера, функция Растригина, функция Экли и функция Букина N.6. Сфера имеет единственный минимум и проверяет скорость сходимости; функции Растригина и Экли содержат множество локальных минимумов и проверяют способность метода не застревать в них; функция Букина N.6 определена только для двух переменных. Сфера, Растригина и Экли запускались в размерностях 2, 10, 30 и 50, что вместе с двумерной функцией Букина N.6 даёт тринадцать задач.

На каждой задаче каждый метод запускался несколько раз с разными начальными значениями генератора случайных чисел, а результаты усреднялись. Параметры методов были подобраны предварительно. Для оценки качества использовались среднее и медианное лучшие значения целевой функции, их разброс, наилучшее значение по всем запускам, а также среднее время счёта. Поскольку у всех рассмотренных функций минимум равен нулю, меньшее найденное значение означает более точный результат.

4.2.2 Сравнение методов на функции с множеством минимумов

Наиболее показательна функция Растригина: у неё много локальных минимумов, и на ней хорошо видна разница между методами. Средние лучшие значения, полученные на ней при разных размерностях, приведены в таблице 1.

Таблица 1 – Среднее лучшее значение на функции Растригина

d	ВВО кл.	ВВО кл. мод.	ВВО син. мод.	ВВО см. мод.	PSO
2	$1,6 \cdot 10^{-1}$	$1,9 \cdot 10^{-2}$	$6,6 \cdot 10^{-3}$	$9,8 \cdot 10^{-3}$	0
10	1,47	0,67	0,65	0,91	5,72
30	14,4	8,08	8,47	8,27	42,5
50	36,0	30,1	28,5	25,6	95,0

Из таблицы видно, что при размерности 2 лучший результат показывает PSO, находя минимум точно. Однако с ростом размерности картина меняет-

ся на противоположную: начиная с десяти переменных, все варианты BBO заметно опережают PSO, и его отрыв растёт. При пятидесяти переменных лучшие варианты BBO дают значение около 25–30, тогда как PSO — около 95. Это объясняется устройством методов: рой частиц быстро стягивается к одной точке и на функции с множеством минимумов застревает в одном из них, тогда как покомпонентный обмен в BBO дольше сохраняет разнообразие популяции. Та же закономерность наглядно показана на рисунке 17.

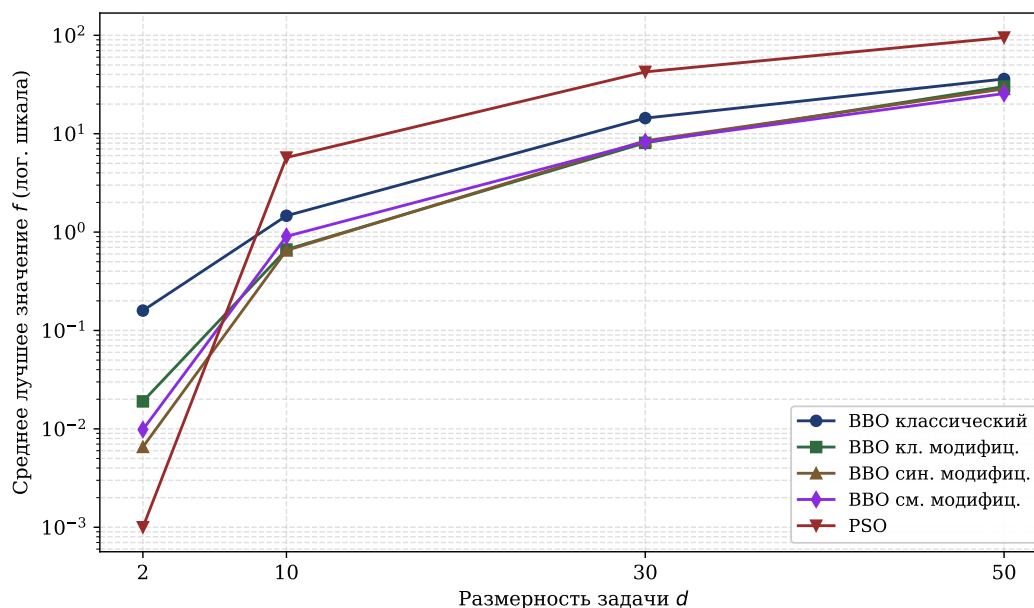


Рисунок 17 – Зависимость среднего лучшего значения на функции Растригина от размерности задачи

Сравнение базовых и модифицированных вариантов показывает, что модификации почти всегда улучшают результат. На функции Растригина любой модифицированный вариант точнее классического во всех размерностях, а в среднем по всем тринадцати задачам модификация оказывается не хуже своего базового варианта в двенадцати-тринадцати случаях из тринадцати. Это подтверждает, что адаптивная мутация, частичный рестарт и локальный поиск действительно помогают методу.

4.2.3 Сравнение методов на разных функциях

Чтобы увидеть общую картину, в таблице 2 приведены средние лучшие значения на трёх функциях при размерности 30 — достаточно высокой, чтобы проявились различия между методами.

Таблица 2 – Среднее лучшее значение на разных функциях при $d = 30$

Метод	Сфера	Растригина	Экли
ВВО классический	$2,3 \cdot 10^{-1}$	14,4	3,69
ВВО кл. модифиц.	$1,8 \cdot 10^{-1}$	8,08	3,35
ВВО син. модифиц.	$1,3 \cdot 10^{-1}$	8,47	2,67
ВВО см. модифиц.	$1,5 \cdot 10^{-1}$	8,27	3,32
PSO	$4,7 \cdot 10^{-5}$	42,5	$2,9 \cdot 10^{-1}$

Таблица подтверждает, что у методов разные сильные стороны. На сфере и функции Экли, рельеф которых ближе к гладкому, PSO заметно точнее: на сфере он находит решение на несколько порядков лучше. А на функции Растригина с её множеством локальных минимумов PSO, наоборот, уступает всем вариантам ВВО в несколько раз. Среди вариантов ВВО различия между моделями миграции невелики, и ни одна из них не оказывается стабильно лучшей на всех функциях; решающий вклад вносят именно модификации, а не модели миграции.

За точность приходится платить временем. На функции Растригина при размерности 30 один запуск классического ВВО занимает в среднем около 0,65 секунды, тогда как модифицированного — около 5 секунд: основное время уходит на локальный поиск, уточняющий решения. Метод роя частиц самый быстрый — около 0,02 секунды на запуск, поскольку его итерация состоит из простых операций над массивами. Таким образом, выбор метода — это выбор между скоростью и точностью на сложном рельефе.

4.2.4 Выводы по результатам

Проведённые эксперименты позволяют сделать несколько выводов. Метод роя частиц предпочтителен на задачах с простым рельефом и при невысокой размерности, где важна скорость сходимости. На задачах с множеством локальных минимумов и высокой размерностью лучше работают варианты биогеографической оптимизации, причём решающую роль играют механизмы адаптации: адаптивная мутация, частичный рестарт и локальный поиск стабильно улучшают результат. Различия же между тремя моделями миграции невелики. Эти результаты согласуются с теоретическими представлениями о поведении методов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы был разработан программный комплекс для исследования и визуализации модификаций алгоритма биогеографической оптимизации. Поставленная во введении цель достигнута, а все связанные с ней задачи решены.

Реализовано серверное вычислительное ядро, поддерживающее семейство из семи методов: классический ВВО, его варианты с синусоидальной и смешанной моделями миграции, три их модификации с адаптивной мутацией, перезапуском при стагнации и локальным поиском, а также метод роя частиц. Алгоритмы запускаются как на четырёх встроенных тестовых функциях, так и на функции, заданной пользователем в виде выражения, причём такое выражение перед вычислением проверяется на безопасность.

Разработан протокол обмена данными поверх постоянного соединения WebSocket. Он поддерживает постановку задачи, передачу прогресса по поколениям в реальном времени, выдачу итогового результата и отмену задачи, а очередь с приоритетами и пул обработчиков позволяют серверу обслуживать несколько клиентов одновременно.

Создано мобильное приложение, через которое пользователь настраивает эксперимент, отправляет задачу на сервер и следит за ходом вычислений. Приложение построено по слоистой схеме, а связь с сервером и хранение истории вынесены в отдельные слои, что упростило его разработку.

Реализованы средства визуализации, наглядно показывающие работу алгоритма: график сходимости, который достраивается в реальном времени, контурная карта и трёхмерная поверхность пространства поиска с возможностью вращения, а также анимация, показывающая динамику популяции. Это позволяет не только увидеть итоговый результат, но и проследить за самим процессом поиска.

Организовано хранение истории экспериментов на устройстве и их сравнение. Сохранённые запуски можно сопоставлять на совмещённом графике сходимости, в таблице показателей и на столбчатой диаграмме, что и составляет основное назначение комплекса.

С помощью построенного комплекса проведены вычислительные эксперименты, в которых семь методов сравнивались на тринадцати задачах разной

структуры и размерности. Эксперименты показали, что метод роя частиц точнее на функциях с простым рельефом и при невысокой размерности, тогда как на функциях с множеством локальных минимумов и высокой размерностью устойчиво выигрывают варианты биогеографической оптимизации. При этом механизмы адаптации почти всегда улучшают результат базового алгоритма, а различия между моделями миграции невелики. Эти результаты подтверждают практическую пользу рассмотренных модификаций.

Комплекс допускает дальнейшее развитие. Семейство методов можно расширить другими метаэвристиками, добавив их в реестр без изменения остального кода. Набор тестовых функций можно дополнить, а заложенные в протокол, но пока не используемые разбиение больших сообщений на части и сжатие — задействовать при работе с большими объёмами данных. Наконец, помимо ручного функционального тестирования клиентскую часть имеет смысл со временем покрыть автоматическими тестами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Карпенко А. П.* Современные алгоритмы поисковой оптимизации. Алгоритмы, вдохновлённые природой : учебное пособие. — 3-е изд. — Москва : Изд-во МГТУ им. Н. Э. Баумана, 2021. — 446 с.
2. *Саймон Д.* Алгоритмы эволюционной оптимизации / пер. А. В. Логунов. — Москва : ДМК Пресс, 2020. — 1002 с.
3. *Карпенко А. П., Синяговская О. А.* Глобальная оптимизация методом биогеографии // Наука и образование (электрон. журн. МГТУ им. Н. Э. Баумана). — 2013. — № 10. — С. 373—398.
4. *Гладков Л. А., Курейчик В. В., Курейчик В. М.* Генетические алгоритмы. — Москва : Физматлит, 2010. — 368 с.
5. *Kennedy J., Eberhart R.* Particle swarm optimization // Proceedings of ICNN'95 — International Conference on Neural Networks. Т. 4. — 1995. — С. 1942—1948.
6. *Пантелеев А. В., Метлицкая Д. В., Алешина Е. А.* Методы глобальной оптимизации. Метаэвристические стратегии и алгоритмы. — Москва : Вузовская книга, 2013. — С. 244.
7. *Карпенко А. П.* Эволюционные операторы популяционных алгоритмов глобальной оптимизации. Опыт систематизации // Математика и математическое моделирование. — 2018. — № 1. — С. 59—89.
8. *Родзин С. И., Скобцов Ю. А., Эль-Хатиб С. А.* Биоэвристики: теория, алгоритмы и приложения. — Чебоксары : Изд-во «Среда», 2019. — 224 с.
9. *Карпенко А. П., Щербакова Н. О., Буланов В. А.* Гибридный алгоритм глобальной оптимизации на основе алгоритмов искусственной иммунной системы и метода роя частиц // Наука и образование (электрон. журн. МГТУ им. Н. Э. Баумана). — 2014. — № 3. — С. 255—271.
10. *Казакова Е. М.* Краткий обзор методов оптимизации на основе роя частиц // Вестник КРАУНЦ. Физико-математические науки. — 2022. — № 2. — С. 150—174.

11. К вопросу эффективности методов и алгоритмов решения оптимизационных задач с учётом специфики целевой функции / Е. Н. Остроух [и др.] // Вестник Донского государственного технического университета. — 2019. — Т. 19, № 1. — С. 81—85.
12. Новикова Е. С., Бекенева Я. А., Бестужев М. П. Методы визуального анализа для мониторинга данных от сенсорных сетей // Известия СПбГ-ЭТУ «ЛЭТИ». — 2020. — № 8/9. — С. 52—60.
13. *CompuServe Incorporated*. GIF89a Graphics Interchange Format Version 89a. — 1990. — 34 с.
14. *Chacón Sartori C., Blum C., Ochoa G.* STNWeb: A new visualization tool for analyzing optimization algorithms // *Software Impacts*. — 2023. — Т. 17. — С. 100558.
15. *Acerbi L.* OptimViz: Visualize optimization algorithms in MATLAB. — URL: <https://github.com/lacerbi/optimviz> (дата обр. 21.05.2026).
16. *Dupont E.* Interactive Visualization of Optimization Algorithms in Deep Learning. — URL: <https://emiliendupont.github.io/2018/01/24/optimization-visualization/> (дата обр. 21.05.2026).
17. *Schäpermeier L., Grimme C., Kerschke P.* To Boldly Show What No One Has Seen Before: A Dashboard for Visualizing Multi-objective Landscapes // *Evolutionary Multi-Criterion Optimization. Lecture Notes in Computer Science*. — 2021. — Т. 12654. — С. 632—645.
18. Олифер В. Г., Олифер Н. А. Компьютерные сети. Принципы, технологии, протоколы. — 5-е изд. — Санкт-Петербург : Питер, 2016. — 992 с.
19. Федюлов А. А. Проектирование архитектуры протокола клиент-серверного взаимодействия web-приложений на основе websocket // Программные системы и вычислительные методы. — 2025. — № 3. — С. 20—30.
20. RFC 6455: The WebSocket Protocol : RFC Editor. — 2011. — URL: <https://www.rfc-editor.org/rfc/rfc6455> (дата обр. 22.05.2026).
21. RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format : RFC Editor. — 2017. — URL: <https://www.rfc-editor.org/rfc/rfc8259> (дата обр. 22.05.2026).

22. Сравнительный анализ форматов сериализации и передачи данных JSON, XML, CBOR и GPB / В. Д. Шульман [и др.] // StudNet. — 2021. — Т. 4, № 7. — С. 1686—1696.
23. NumPy documentation : NumPy documentation. — URL: <https://numpy.org/doc/stable/> (дата обр. 22.05.2026).
24. *Яворски М., Зиаде Т.* Python. Лучшие практики и инструменты. — Санкт-Петербург : Питер, 2024. — 592 с.
25. FastAPI : FastAPI documentation. — URL: <https://fastapi.tiangolo.com/> (дата обр. 22.05.2026).
26. json — JSON encoder and decoder : Python 3 documentation. — URL: <https://docs.python.org/3/library/json.html> (дата обр. 09.04.2026).
27. dart:convert : Dart documentation. — URL: <https://dart.dev/libraries/dart-convert> (дата обр. 09.04.2026).
28. *Рябоконь О. С., Кукарцев В. В.* Новый язык структурного веб-программирования Dart // Актуальные проблемы авиации и космонавтики. — 2013. — Т. 1, № 9. — С. 436—437.
29. *Камышев А. О., Зотин А. Г.* Особенности использования фреймворка Flutter при разработке кроссплатформенных приложений // Актуальные проблемы авиации и космонавтики: сб. материалов VI Междунар. науч.-практ. конф. Т. 2. — Красноярск : СибГУ им. М. Ф. Решетнёва, 2020. — С. 138—140.
30. pytest: helps you write better programs : pytest documentation. — URL: <https://docs.pytest.org/en/stable/> (дата обр. 22.05.2026).

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

Руководство пользователя

Руководство описывает работу с мобильным приложением комплекса — от запуска сервера до анализа результатов. Предполагается, что серверная часть уже запущена и доступна по сети, а мобильное приложение установлено на устройство.

Перед началом работы необходимо запустить сервер на компьютере, доступном по сети. Сервер начинает принимать подключения по указанному адресу и порту. После этого работа ведётся через мобильное приложение в следующем порядке.

1. Подключение к серверу. При запуске приложение открывает экран подключения. В поля вводятся адрес и порт сервера, после чего нажимается кнопка подключения. Если соединение установлено, приложение переходит к экрану настройки; иначе под кнопкой выводится сообщение об ошибке, и адрес следует проверить. Внешний вид экрана приведён на рисунке 10.
2. Настройка эксперимента. На экране настройки выбирается целевая функция — встроенная или заданная собственным выражением, — задаются размерность и границы области поиска, выбирается метод оптимизации и его параметры. Для большинства параметров уже подставлены разумные значения по умолчанию, поэтому при первом знакомстве достаточно выбрать метод и функцию. Экран настройки показан на рисунке 11.
3. Запуск и наблюдение за вычислением. После нажатия кнопки запуска приложение переходит к экрану выполнения, где отображаются номер текущего поколения, лучшее найденное значение и график сходимости, обновляемый в реальном времени. При необходимости вычисление можно прервать кнопкой отмены. Этот экран показан на рисунке 12.

4. Просмотр результата. По завершении вычисления открывается экран результата с лучшим решением, показателями качества, графиком сходимости и контурной картой пространства поиска. Здесь же доступны построение анимации и переход к трёхмерной визуализации, которую можно вращать движением пальца. Вид экрана результата приведён на рисунках 13–15.
5. Работа с историей и сравнение. Каждый завершённый эксперимент сохраняется в историю, доступную с экрана настройки. В истории запуски можно фильтровать, открывать повторно и отбирать несколько штук для сравнения. На экране сравнения выбранные запуски выводятся вместе — на совмещённом графике сходимости, в таблице показателей и на столбчатой диаграмме. Экраны истории и сравнения показаны на рисунке 16.

При потере соединения с сервером приложение пытается восстановить его автоматически. Если сервер недоступен длительное время, следует проверить, запущен ли он, и повторить подключение вручную с экрана подключения.